



תיק פרויקט עבור המשחק:
"TopGun – The Danger Zone"
עבודתו של נועם בן סעדון.
מורה מלווה: יוסי זהבי.

ניהול עולם תלת-ממדי באסמבלי 8086, 16 ביט, TASM IDEAL.



ת.ז: 334945474

נועם בן סעדון | תיכון הרצוג כפר סבא | כיתה י'4

מורה מלווה: יוסי זהבי.

קישור לגיטהאב

תוכן עניינים

3	מבוא
4-5	קבצים נלווים
6	סביבת הפיתוח, העבודה, וההרצה
7	נושא העבודה
8	אופן ההפעלה
9-10	גרסאות מערכת
11-13	תיעוד והסבר הפתרון
14-24	תרשימי זרימה
25-78	רשימת הפעולות
79-83	דוגמאות הרצה
84	סיכום אישי

מבוא

עבור פרויקט גמר בו התבקשנו לכתוב באסמבלי 8086 (ראו פרטים בהמשך), החלטתי לכתוב משחק, אותו כיניתי¹ **"TopGun – The Danger Zone"**, העוסק בדימוי לא-ריאליסטי של לחימה אווירית בין מטוסי קרב בעולם תלת ממדי.

המשחק הוא סימולטור קרבות אוויר "ארקייד". השחקן שולט במטוס קרב מסוג F-14 Tomcat, ונדרש להשמיד גלים של מטוסי אויב ולהתחמק מטילים לאורך עולם אינסופי.

הז'אנר הוא Flight Combat Simulator, פיתוח מז'אנר המשחק ממנו לקחתי השארה – After Burner (NES). הפיתוח נובע מכך שניתן לזוז גם שמאלה וימינה באופן חופשי בעולם, ולא מחויבים להישאר קדימה. ניתן לכנותו "6DoF" פשוט, ובעתיד, בגססאות הבאות, בתקווה "Free-Roaming".

מטרת השחקן היא פשוטה – לשרוד ולהשמיד.

הישרדות: על השחקן להתחמק מטילי אויב.

תקיפה: על השחקן להשמיד מספר רב ביותר של מטוסי אויב.

התקדמות: המשחק אינסופי, ובכך נתון לשחקן עצמו עד שיחליט ששיחק מספיק.

הסביבה הטכנית היא DOSBOX, במצב VGA 13h עם מסך 320x200.

הפרויקט עוסק בתכנון ומימוש של סימולטור טיסה אינטראקטיבי, ונכתב כולו בשפת Assembly (ארכיטקטורת x86).

הפרויקט בנוי, כראוי עבור אסמבלי 8086, עם חלוקה לסגמנט נתונים וקוד בקובץ ה-ASM, וגם כתיבה וקריאה מ-ES עבור הגרפיקה.

עבור הפרויקט, נאלצתי להתמודד עם בעיות ולנהל את הזיכרון RAM, את המחסנית, וכל רג'יסטר ורג'יסטר ב-CPU. נאלצתי כמו כן לשמור על STACK FRAME תקין בכל פעולה בשיטה הסטנדרטית, ובעזרת משאבים מוגבלים, לבנות משחק באיכות גבוהה.

הפרויקט נכתב במודל IDEAL SMALL של אסמבלר מעבד 8086 של INTEL שיצא בשנות ה-80.

כל הקוד בשפת האסמבלר נמצא בקובץ אותו כיניתי² **"MAIN"** בהיותו החלק העיקרי ביותר בתוכנית. אף על פי כן, לקובץ זה נלווים קבצים רבים, כפי שניתן לראות בעמוד הבא.

תיק פרויקט זה עוסק בתיעוד ותכנון הפרויקט במלאו. בתיק ניתן למצוא תרשימי זרימה, הסבר עבור כל פעולה ופעולה בתוכנית, תיעוד עבור הפתרון, גרסאות עתידיות, ולקרוא על סביבת העבודה עצמה ואופן הפעלת המשחק עצמו, יחד עם תמונות היישר מן המשחק.

במהלך הפרויקט, עבור העברת נתונים נוחה בין שני מחשבים עיקריים עליהם עבדתי, השתמשתי ב-GIT וב-GITHUB. למדתי להשתמש כך גם בסביבת עבודה מקצועית, למדתי על Merge Conflict, ספריות, והרבה על GITHUB Open Source Code.

¹ שם העבודה.

² שם הקובץ.

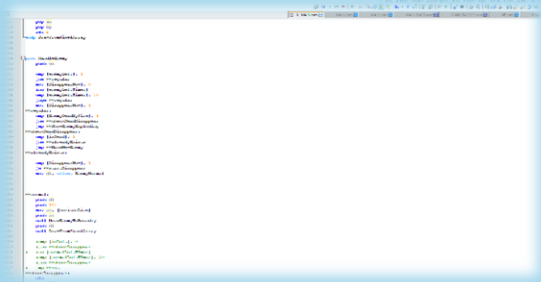
קבצים נלווים

שם הקובץ	סיומת וסוג הקובץ	תיאור הקובץ	תפקיד הקובץ בתוכנית
E_EB.bmp	.bmp תמונה	תמונה של מטוס מפוצץ לחלוטין.	מטוס אויב בשלב השני לפיצוץ לאחר פגיעת יריית השחקן.
E_ES.bmp	.bmp תמונה	תמונה של מטוס מפוצץ למחצה (נראה כבתחילת הפיצוץ)	מטוס אויב בשלב הראשון לפיצוץ לאחר פגיעת יריית השחקן.
explo.bmp	.bmp תמונה	תמונה של פיצוץ.	כשמטוס השחקן מיורט / מתרסק תמונה זו תוצג.
F_D.bmp	.bmp תמונה	מטוס השחקן שהוא פונה מטה.	עבור ירידה למטה במשחק.
F_L.bmp	.bmp תמונה	מטוס השחקן שהוא פונה שמאלה.	עבור פנייה שמאלה במשחק.
F_R.bmp	.bmp תמונה	מטוס השחקן שהוא פונה ימינה.	עבור פנייה ימינה במשחק.
F_U.bmp	.bmp תמונה	מטוס השחקן שהוא פונה מעלה.	עבור עלייה למעלה במשחק.
FIGHTHER.bmp	.bmp תמונה	מטוס השחקן שהוא ישר וניטרלי לחלוטין.	המטוס כשלא בוחרים לזוז לשום כיוון.
G_O.bmp	.bmp תמונה	תמונה אשר מוצגת כשהשחקן מפסיד.	עבור שליחת מסר ההפסד בצורה חד משמעית.
L_2.bmp	.bmp תמונה	תמונה של מטוס וכיתוב "Press Any Key".	מסך פתיחה.
L1.bmp	.bmp תמונה	תמונה של מטוס על שביל המראה.	CUTSCENE.
L2.bmp	.bmp תמונה	תמונה של מפקדה של חיל האוויר.	CUTSCENE.
L3.bmp	.bmp תמונה	תמונה של מטוס חונה עם צוות סביבו.	CUTSCENE.
P_XS.bmp, P_S.bmp, P_M.bmp, P_L.bmp	.bmp תמונה	מספר תמונות של המטוס בגדלים שונים.	עבור סצנת המופיעה לאחר ה-CUTSCENE.
LOADING.bmp	.bmp תמונה	תמונה של מטוס עם הכיתוב Exit ו Play.	מסך ראשי.
M_E.bmp	.bmp תמונה	פיצוץ גדול	השלב השני בפיצוץ מטוס אויב.
M_E2.bmp	.bmp תמונה	התחלתו של פיצוץ	השלב הראשון בפיצוץ מטוס אויב.
EXPL.raw	.raw קובץ פשוט לא רק סאונד, המכיל דגימות גלי קול.	סאונד פיצוץ במים.	כשהשחקן מתרסק למים.
INST_1.raw	.raw קובץ פשוט לא רק סאונד, המכיל דגימות גלי קול.	חלק ראשון של CUTSCENE	בתחילת המשחק, עבור הסיפור ב-CUTSCENE.
INST_2.raw	.raw קובץ פשוט לא רק סאונד, המכיל דגימות גלי קול.	חלק שני של CUTSCENE	בתחילת המשחק, עבור הסיפור ב-CUTSCENE.
INST_3.raw	.raw קובץ פשוט לא רק סאונד, המכיל דגימות גלי קול.	חלק שלישי של CUTSCENE	בתחילת המשחק, עבור הסיפור ב-CUTSCENE.
TDZ.raw	.raw קובץ פשוט לא רק סאונד, המכיל דגימות גלי קול.	שיר	מוזיקה במסך הפתיחה.
TMBA.raw	.raw קובץ פשוט, לא במיוחד עבור סאונד, המכיל דגימות גלי קול.	שיר	מוזיקה בתפריט הראשי.

שם הקובץ	סיומת וסוג הקובץ	תיאור הקובץ	תפקיד הקובץ בתוכנית
RW.bmp	.bmp תמונה	תמונה של נושאת מטוסים בלי רקע.	לאחר ניגון ה-CUTSCENE הראשוני, תופיע סצנה בה ממריא המטוס מנושאת המטוסים.
SPD.bmp	.bmp תמונה	תמונה של טקסט "SPEED:".	עבור מד המהירות.
ALT.bmp	.bmp תמונה	תמונה של טקסט "ALTITUDE:".	עבור מד הגובה.
1.bat	.bat Batch File – קובץ אשר מפורש על ידי המפרש של מערכת ההפעלה ומורץ (כתובות הוראות).	מקמפל ומלנקז את הקובץ MAIN עם הערות עבור TD.	קימפול התוכנית לפני הפעלה מחדש.
RUN.bat	.bat Batch File – קובץ אשר מפורש על ידי המפרש של מערכת ההפעלה ומורץ (כתובות הוראות).	קובע הגדרה (כמות cycles) עבור DOSBOX, ולאחר מכן מריץ את התוכנית MAIN. הערה: לא מקמפל ומלנקז.	קביעת כמות ה-CYCLES עבור ריצה נוחה וזורמת של התוכנית. גם מריץ את התוכנית. בעיקרון – ניתן לעשות באופן ידני בקלות.
DPMI16BI.OVL	.ovl Overlay File – מכיל פונקציות או נתונים שצריך רק מדי פעם.	נחיצות של TD עבור כל קוד.	משומש על ידי סביבת העבודה באופן אוטומטי.
TASM.EXE	.exe Executable File – תוכנה שאנו מריצים.	תוכנה אשר מקמפלת את הקוד.	קימפול MAIN.
TD.EXE	.exe Executable File – תוכנה שאנו מריצים.	תוכנה אשר מריצה ומאפשרת לדבג את הקוד.	דיבוג MAIN.
TDCONFIG.TD	.td Turbo Debugger configuration file - קובץ נלווה המשמש את הדיבאגר	נחיצות עבור TD.	נחוץ עבור TD. משומש על ידי סביבת העבודה.
TLIB.EXE	.exe Executable File – תוכנה שמורצת.	מנהל קבצים, נחיצות עבור סביבת העבודה.	משומש על ידי סביבת העבודה.
TLINK.EXE	.exe Executable File – תוכנה שאנו מריצים.	תוכנה אשר מלנקזת את הקוד.	לינקז' MAIN.
RTM.EXE	.exe Executable File – תוכנה שאנו מריצים.	תוכנת נחיצות של TLINK.	קשור בלינקז'.
MAIN.asm	.asm קובץ מקור עבור ספת אסמבלר.	הקובץ בו נמצא כל הקוד.	התוכנית עצמה.
READ_ME.txt	.txt קובץ טקסט	קובץ עם מידע חיוני על הקבצים.	עבור הבוחן.

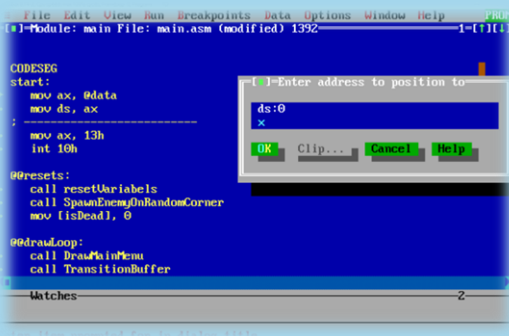
סביבת הפיתוח, העבודה, וההרצה

סביבת הפיתוח של הפרויקט הייתה בתוכנה חינוכית בשם "Notepad++", עורך טקסט פחות "חכם" מתוכנות כמו Word, אבל יותר נוח ומותאם לתכנות באסמבלי מתוכנות כמו Notepad ו- Visual Studio (for ex. Community).

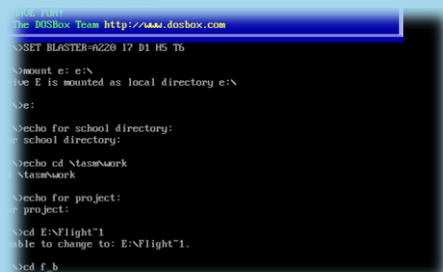


סביבת העבודה של הפרויקט הייתה Turbo Assembler,

וכללה את TASM ו- TLINK גם כן. בכדי ליצור את קובץ ההפעלה (EXE), יש ראשית לשמור את קובץ ה- ASM שערכנו ב- "Notepad++". לאחר מכן, ננווט לנתיב הקובץ דרך הכרת הכונן לסביבת ההרצה וניווט לתיקייה (ראו "סביבת ההרצה") אחריו, מגיע שלב ההידור (Assembly). בשלב זה, אנו הופכים את קוד המקור לקובץ OBJ. הפקודה שיש להריץ הינה `tasm /zi file_name`. כאן נראה טעויות קימפול אם קיימות. שימו לב – חשוב ששם התיקיות והקבצים ב- PATH יהיו עם מקסימום של 8 אותיות! (אנגלית). דגל ה- `/zi` שאנו מוספים מוסיף "הערות" עבור TD, אשר יקרא אותם במידה ונרצה לדבג. בשלב זה, נרצה לעבור לשלב ה- "לינקינג" או קישור (Linking). נריץ את הפקודה `tlink /v file_name`. שוב, דגל ה- `/v` הוא עבור הדיבוג לאחר מכן. כעת, יצרנו את קובץ ה- EXE, שנוכל להריץ עם הקשת שם הקובץ. אם נרצה לדבג, נכתוב `td file_name` ולא רק את שם הקובץ, וכך נפתח את התוכנה Turbo Debugger, טעונה על הקובץ שלנו. קיים ממשק גרפי, המאפשר הרצת הקוד



בשלבים, יחד עם צפייה בסגמנטים השונים. מתקדמים עם F8 עבור Step Over (לא להיכנס לקריאות לפעולות), ועם F9 עבור Step Into. ניתן לסמן Break Points עם F2, וכמון כן עומדים בפני המתכת כלים רבים נוספים. עבודה עם TD לרוב תכלול הרצת 2 הפקודות הראשונות באופן מיידי, שמעבירות ל- ds את הכתובת של הנתונים, ולאחר מכן עוברים למקום הרצוי או ל- offset בסגמנט הנתונים דרך `Ctrl+G`, ולאחר מכן `ds:0`, עבור תצוגה נוחה של הקוד אל מול סגמנט הנתונים. במוד גרפי, לרוב מדבגים דרך כתיבת למסך, לא דרך TD.



סביבת ההרצה של הפרויקט היא דרך מנוע דימוי DOS, "DOSBOX",

שמדמה בתוך המחשב שלנו מעין מחשב נוסף, שרץ על מעבד 8086. הכל רץ בתוך סביבה זו, והיא חיונית משום שבלעדיה (כמובן שקיימות חלופות) לא ניתן להריץ את הקובץ. בכדי לנווט ליעד שלנו בתוכנה זו, ראשית כל נריץ את הפקודה `mount e: e:\` (אם נרצה עבור כונן e). בפקודה זו אנו "נותנים שם" לכונן e (עבור נוחות) ולאחר מכן נוכל פשוט לכתוב את השם בכדי לנווט לכונן. שימו לב – ה"שם" חייב לעקוב אחר syntax של `paths`. לאחר מכן נוכל לנווט דרך `cd target_directory`, או `cd ..` אם נרצה לדוגמה לנווט אחורה. קיימות פקודות נוספות, וניתן לשנות כל מיני הגדרות של dosbox דרך קובץ ה- configuration שמגיע בהתקנה.

נושא העבודה

המשחק שלי, "TopGun – The Danger Zone" הוא משחק המדמה לחימה אווירית בין טייס חיל האוויר הישראלי שהצטרף לנושאת מטוסים של ארה"ב, שמספקת תמיכה בשקט. על הטייס ליירט את מטוסי האויבים. המשחק נלקח בהשראת המשחק "After Burner" של SEGA, אך בניגוד למשחק זה, התנועה בעולם אפשרית כתנועה מרחבית בעולם עצמו, ולא בשטח מוגדר במסך. ניתן לפתוח MAIN MENU עם ESC.

פרויקט זה מהווה מחקר מעשי ויישומי בתכנות מערכות וגרפיקה ממוחשבת בזמן אמת, תוך עבודה ישירה מול רכיבי החומרה של ארכיטקטורת x86 בניגוד לפיתוח מודרני הנשען על שכבות אבסטרקציה וספריות מוכנות, פרויקט זה נבנה מתוך ה"וואקום" של שפת ה-Assembly, מה שאילץ פיתוח של פתרונות אלגוריתמיים ייחודיים לניהול זיכרון, עיבוד תמונה ותקשורת בין-תהליכית. לב המערכת נשען על מנוע רינדור גרפי המממש את טכניקת ה Double Buffering – באופן ידני. האתגר ההנדסי המרכזי היה התגברות על צוואר הבקבוק של כתיבה לזיכרון הווידאו (VGA) על ידי הקצאת סגמנט זיכרון ייעודי המשמש כ"באפר" אחורי, המערכת מבצעת את כל חישובי הלוגיקה והציור בנפרד מהתצוגה, ורק לאחר השלמת הפריים מתבצעת העתקה מסיבית ומהירה של בלוק נתונים לזיכרון הווידאו. גישה זו מאפשרת השגת קצב רענון גבוה ותנועה חלקה ללא הבהובים, המדמה ביצועים של מנועים גרפיים מסחריים.

המורכבות האלגוריתמית של הפרויקט באה לידי ביטוי במערכת הטיסה הדינמית. כדי ליצור תחושת מרחב, פותח מנגנון החלפת מצבים ויזואלי המגיב לקלט המשתמש דרך גישה ישירה לפורט המקלדת (I/O Port 60h) להסתמך על שירותי מערכת ההפעלה האיטיים, המערכת דוגמת את ה Scan Codes – של המעבד ישירות, מה שמאפשר תגובתיות מיידיית וביצוע פעולות מקביליות, כמו תמרון המטוס תוך כדי ירי.

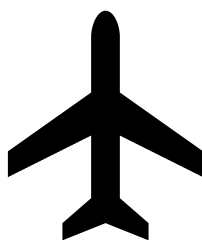
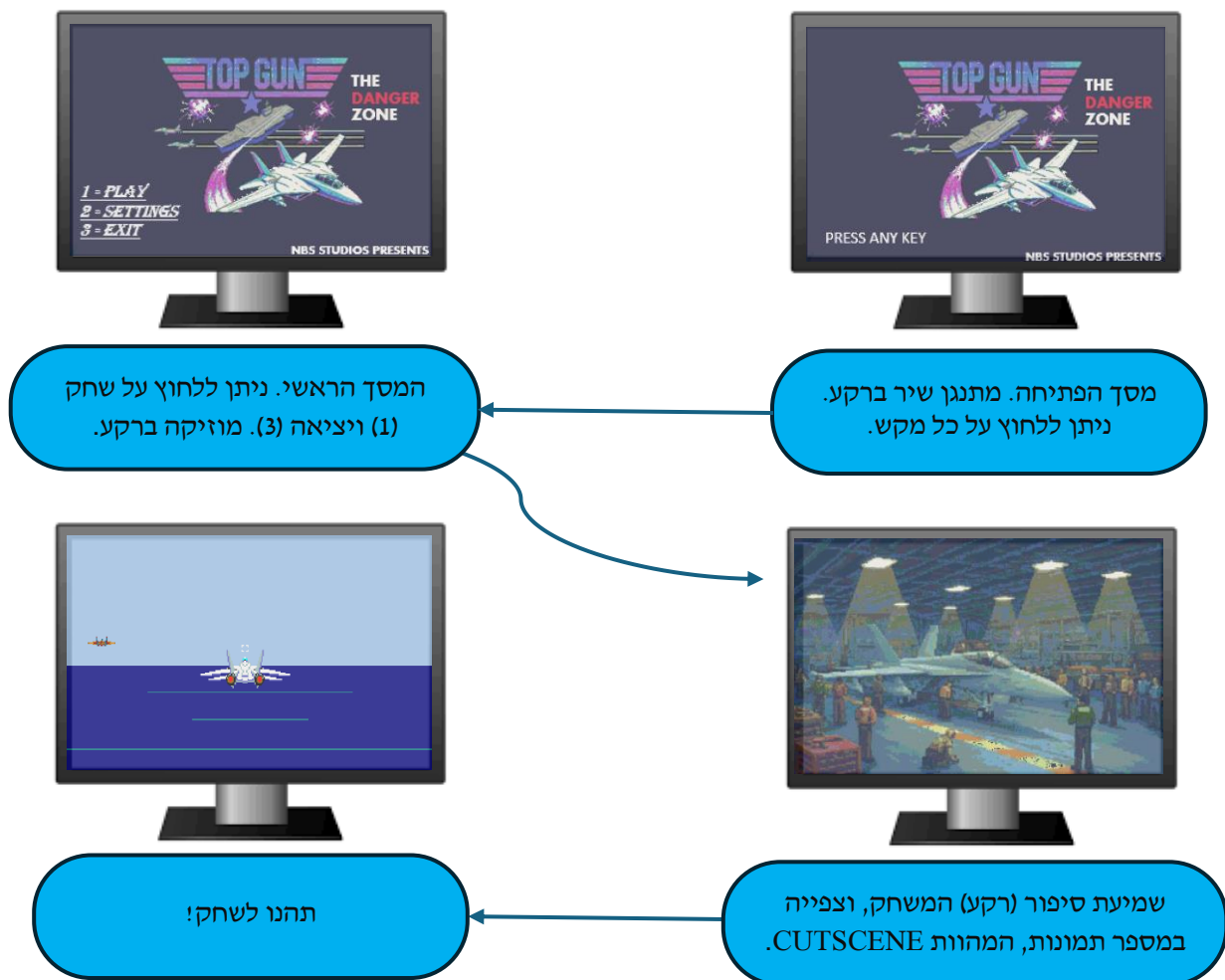
בנוסף, משולב רכיב שמע המפעיל קבצי קול (RAW) לסנכרון אפקטים קוליים עם המתרחש במסך. בנוסף, הפרויקט מנצל את פסיקת הטיימר עבור ניגון רעשי רקע ומוזיקת רקע. הפרויקט כולל ניהול אויבים (Enemies) בעלי רמות קרבה שונות (Close/Far), מערכת זיהוי פגיעות (Collisions) וניהול אירועים מבוסס מיקום וטיימרים. בפרויקט מופיעות פעולות הן בשטיה הסטנדרטית, והן דרך העברה והחזרה באוגרי הזיכרון.

התנועה מתבצעת בעזרת המקשים W A S D. W – תנועה למעלה. A – תנועה שמאלה. S – תנועה מטה. D – ימינה. ניתן לירות בעזרת מקש ה-J במקלדת, אך האפשרות לירות מתחדשת רק כאשר הירייה האחרונה נעלמת.

אויבים מזדמנים באופן רנדומלי בצד ימין / שמאל, וטסים אל עבר מרכז המסך. לאחר מכן, האויב נראה כמתרחק עם הזמן. ועף אל קצה המרכז הנגדי אליו (מעלה).

אופן ההפעלה

חשוב: על כל הקבצים המורדים להיות בתוך אותו תיקייה.
עם הרצת התוכנית, המערכת מבצעת סדרת פעולות אתחול הכוללות מעבר למצב גרפי 13h וטעינת נכסי התפריט הראשי. המשתמש פוגש מסך פתיחה הממתין לכל קלט, ושיר מתנגן ברקע. לחיצה על כל מקש תעביר את המערכת ממצב "המתנה" למצב "מסך ראשי". לאחר מכן, נתונות האופציות משחק, ויציאה. לחיצה על 1 תוביל לאת המשתמש למשחק, ואילו 3 ליציאה. לחיצה על 2 תוביל את המשתמש, בעתיד, למסך ההגדרות. לאחר לחיצה על אחד, ייטען מצב "CUTSCENE" המציג את הסיפור שעוברת הדמות במשחק. ניתן לדלג על הלחיצה CUTSCENE בלחיצה על רווח. לאחר מכן, המשתמש נזרק ישירות למשחק, ויכול להתחיל לשחק!



גרסאות המערכת

בפריקט זה, התחלתי מהצגת ריבוע על המסך, עברתי לתיקון הבהובים וסאונד, ולאחר מכן גם עסקתי בתלת ממד, טעינת תמונות BMP, פגיעה במטרה, יריות, זיהוי התנגשות, וכן הלאה. להלן מה שנכלל בגרסה זו:

1. מנוע רינדור עם באפר כפול – בכדי לפתור את בעיית הריצוד, המערכת עברה שדרוג משמעותי מכתובה ישירה לזיכרון המסך, ובמקום – כותבת לסגמנט שמוקדש כולו עבור כתיבה למסך. משתמשת בסגמנט נוסף, כאשר בכל סוף איטרציית לולאה מעתיקים את כל מה שנכתב לסגמנט זה לזיכרון המסך.
2. טעינת תמונות BMP למסך.
3. מערכת סאונד – שימוש בקבצי RAW לטעינת שירים אמיתיים ודיבור, וכמו כן, עבור מוזיקת הרקע, שימוש בPC SPEAKER, להוצאת תדרי הרץ.
4. עולם "תלת ממדי" – תנועה של גלים אשר יוצרת את האשליה של עולם תלת ממדי, בו ניתן לזוז למעלה, למטה, ימינה ושמאלה, להאיץ ולהאט כלפי קדימה.
5. עולם אינסופי – לאחר פניות ארוכות ימינה / שמאלה, האובייקטים (גלים) שראינו קודם לכן – חוזרים.
6. מקלדת אסינכרונית המאפשרת קבלת מידע על הקשה על 2 מקשים בו זמנית וזמן תגובה מהיר במיוחד.
7. אויבים – האויבים מתרחקים ו"בורחים" מהשחקן קדימה, על המשחקן ליירט אותם.
8. מערכת יריות של השחקן המוגבלת בCOOLDOWN, ומערכת טילים שבאים אל השחקן במטרה ליירט אותו, מהם עליו להתחמק במהלך המשחק.
9. שיטות האטה של אירועי עולם שונים (כמו זימון טיל) – בעזרת SHL כל פריים ובדיקת CF.
10. טעינת SPRITES מתוך הזיכרון (מערכים של נקודות וצבעים) אל המסך.
11. פעולות דינמיות בשיטה הסטנדרטית.
12. ייצוג גלים דרך הוספות "רעש" רנדומלי לקווים ישרים אותם מצייר האלגוריתם של Bresenham.
13. חישוב מיקום במסך דו מימדי לפי קואורדינטה תלת ממדית.
14. HUD על המסך המציג מד מהירות וגובה.

בגרסאות הבאות, ארצה לפתח את ההבאים :

1. מודל תלת ממדי אמיתי – במקום להסתמך על Sprites מוכנים, המערכת תחשב קואורדינטות Z ותבצע סיבוב של אובייקטים במרחב באמצעות מטריצות סיבוב וטבלאות סינוסים/קוסינוסים באסמבלי. זה ידרוש שימוש גם בFPU.
2. מימוש "בינה מלאכותית" חכמה לאויבים – ככל שהזמן עובר, אויבים יצברו נקודות כשהצליחו להתחמק, ועם משתנה רנדומלי לגיוון, ישארו במסלולים בטוחים יותר, מה שיקשה על השחקן ליירט.
3. מערכת ניהול שלבים דינמית – פיתוח מנגנון הטוען מתוך קובץ טקסט חיצוני את מבנה השלב (מיקומי אויבים, תזמונים, ואירועים), מה שיפריד לחלוטין בין קוד המנוע לתוכן המשחק.
4. משחק "אונליין" – מימוש המשחק כך שיהיה זמין עבור מספר משתמשים באותו זמן, באותו "לובי". דרך שיתוף המידע בשרת ברשת והשוואה למשתמשים אחרים הנמצאים באותו "לובי" אך לא באותו עולם (השוואת נקודות בשלבים זהים).
5. בהסתמך על מערכת "אונליין" קיימת – פיתוח מצב לחימה אווירית בין משתמשים. המשתמשים נמצאים לא רק באותו "לובי", אלא גם באותו עולם, ויכולים להילחם זה בזה.
6. מימוש מנוע פיזיקה – הפיכת המשחק לריאליסטי יותר מבחינת הפיזיקה, והשפעות המהירות והגובה על שאר העולם.
7. מנוע חלקיקים – הוספת מנוע חלקיקים היאפשר לראות פרטקילים של יריות ופיצוצים, בנוסף למים שיוזזו עם התקרבות המטוס. כמו כן, הוספת צללים, השתקפויות, ומימוש אלגוריתם שיצור אשליה שנראית כרעידת המסך עם פיצוצים.
8. הוספת CUTSCENES וסצנות נוספות.
9. הוספת מכניקות לחימה חדשות – הפלת פצצות, טילים מונחים, וכן הלאה.
10. שיפור מוזיקת הרקע.
11. יצירת מפקדה תלת ממדית בה ניתן ללכת בעזרת מודל Ray Casting. כבר יש ברשותי את הקוד הנדרש עבור עולם תלת ממדי כזה, אך הוא לא מעוצב די עבור הפרויקט.
12. מצב משחק – חייל במפקדה, שרואה את המשימה ממעל.
13. שלב בו ניתן להסתמך על כלי הטיסה בלבד, בלי ראיית העולם, מתוך מושב הטייס (מד מהירות, גובה, מפות, מפקדה, בקיצור – IFR).
14. סצנת סיום שתידמה כסרטון דרך החלפת פריימים (תמונות) מהר במיוחד, שתשים את הפנים של השחקן על הדמות (בעזרת תוכנת עזר בפייתון שתצלם את הראש של השחקן דרך המצלמה ותחתוך את הפנים) ומראה את החייל חוזר מהמשימה.
15. אפשרות לחזור אחורה בזמן, עד 5 שניות, בלחיצה על כפתור REWIND.
16. שלבים מפותחים עם הרים מהם יש לחמוק, גשרים, וכד'.

תיעוד והסבר הפתרון

מערכת זו אינה תוכננה להיות דף הוראות שמבצע רצף פעולות, אלא מן היסוד תוכננה כמנוע משחקים מונחה אירועים. בסביבת הפיתוח של אסמבלי 8086 לא קיימת מותרות של ספריות מוכנות או מערכת שמנהלת עבורי את המשאבים. כלומר - כל פיקסל על המסך, כל בייט בזיכרון וכל פקודה למעבד הם תכנוני, וניתן לדעת בדיוק מה קורה לפי כל שורת קוד – לדעתי מותרות, שלא קיימת בשפות עיליות. להלן פירוט מעמיק של ארכיטקטורת המערכת, אופן החזקת הנתונים והאלגוריתמים המניעים אותה:

הבעיה: השוק היום מלא בתוכן שחוזר על עצמו, משחקים מלאים בבקשות רבות מהמערכת ובבעיות פוטנציאליות של חדירה לפרטיות ותנאי משתמש מסובכים. בנוסף לכך, כל המשחקים חוזרים על עצמם וקשים להסתגלות אם לא קונים שדרוגים בכסף.

הפתרון: משחק הכולל לוגיקה של מתח והרפיה. קיים אתגר קצר, מהיר ואינטנסיבי שדורש ריכוז שלם ומוחלט, ואחריו מגיע התגמול (הישרדות). השחקן נמצא במצב זורם שגורם לו לרצות להמשיך לשחק במשחק.

ראשית הכל – המסך. התוכנית משתמש ב-VGA מצב 13h עבור הצגה על המסך, דרך כתיבה ב-ES החל מהכתובת 0A000h.

מטרת התוכנית היא להריץ משחק שבו השחקן מתקדם בעולם תלת ממדי, ועליו ליירט אויבים ולהתחמק מטילים.

התוכנית גם צריכה להגיב לקלט מהמשתמש. לפי לחיצה, על התוכנית להגיב בצורה הנכונה כדי להציג את הפלט הנכון על גבי המסך. מלבד הזזת השחקן (אשליה – הגלים הם אלו שזזים), על התוכנית גם לסנכרן את התנועה עם שאר האובייקטים, כמו האויבים. הקליטה מתבצעת דרך המקלדת באמצעות פסיקה אסינכרונית. העולם מצטייר, האויבים זזים לקראת / הרחק מהאויב, טיל עלול לחלוף, מוזיקת רקע מתנגנת ועוד... בסוף איטרציה, לאחר ניהול אירועים רבים נוספים (כפי שניתן לראות בתרשים הזרימה או בתיאור האלגוריתם) כל המידע מועתק למסך עבור השחקן.

התוכנית מחולקת לשני חלקים (סגמנטים) חשובים.

-CODESEG

בסגמנט הקוד, מופיעות כל הפרוצדורות של התוכנית, וכמו כן גם קוד המופיעה לפני הפרוצדורות כמו איפוסים בתחילת התוכנית, מסך הפתיחה, הלולאה הראשית, וכד' (הסבר בהמשך). סגמנט זה הוא בעצם ספר ההוראות. הוא אומר למחשב בדיוק מה לעשות בכל שורה, מבלי שהמחשב "רואה" את התמונה הגדולה. בלי הקוד, אין משחק.

-DATASEG

בסגמנט הנתונים נמצאים משתני התוכנית. בין דגלים, לשמות קבצים, וכמו כן גם מערכים, לדוגמה, Sprites של דמות האויב במרחקים שונים ושמות של קבצים אשר פותחים בזמן ריצה. סגמנט זה מאחסן את כל הנתונים החשובים שיש לשמור עבור טווח ארוך, בין אם הם משתנים (מלשון שינוי), ובין אם לא. בלי סגמנט עבור ניהול הזיכרון (ממנו ניתן להמנע באופן טכני) העבודה תהיה מוגבלת לחלוטין, ולכן סגמנט זה, אשר לוקח חלק משמעותי משורות הקוד בפרויקט זה, כל כך חשוב. בפירוט, ב-DATASEG מוחזקים הנתונים העיקריים של התוכנית (ואלו הסוגים העיקריים):

1. דגלים: דגלים דלוקים (בסטנדרט 1) או כבויים (בסטנדרט 0) שמסמנים האם אירוע קרה או לא.
2. מבני נתונים: מערכים ארוכים, חלקים מייצגים רשימה של דגלים (מערך של מקשים), נקודות על המסך (SPRITES), נקודות קצה תלת ממדיות, שמות קבצים (שצריכים להסתיים ב0 לסימון), וכד'.
3. משתנים עזר שמורים: משתנים אשר משתנים (מלשון שינוי) בין תוכנית, ועל כן יש לשמור אותם בכדי להעביר אותם בין תוכניות שלא בהכרח קוראות אחת לשניה (מה שהופך שימוש במחסנית לבעייתי).
4. משתני עזר זמניים: משתנים פשוטים שלרוב מאופסים לאחר שימוש, כמו שיטות המתנה שאינן WAITING BUSY עבור המעבד (shl וחיפוש אחר CF).

התוכנית גם משתמשת לא פעם ב-MACROS, שמות שניתן להעניק לקטעי קוד או לנתונים, שאינם פעולות, אלא במהלך הקמפול מוחלפים (היכן שכתוב השם, שם יופיעו הנתונים המתאימים).

בנוסף לכך, התוכנית תלויה גם ב-Stack, שבעזרתו ניתן להעביר פרמטרים בין פעולות. עובד במנגנון LIFO (Last In First Out).

כמו כן, התוכנית תלויה גם ב-secondBufferSeg. אני כותב את כל המידע שיש להציג למסך לסגמנט זה, שהוגדר עבור מניעת ריצודים. הוא מכיל 64,000 בתים עבור מסך VGA 320x200. בכל סוף איטרציה, אני מעתיק מסגמנט זה אל המסך.

חשוב לציין שגרפיקת העולם מתבססת על מספר שיטות עיקריות:

1. תמונות: תמונות אשר מצוירות או לרגעים ספורים (פיצוץ מטוס אויב) או באופן מתמיד לאורך כל המשחק (כמו תמונת מטוס השחקן).
 2. מערכים מוכנים מראש בשיטת "dw X, Y, Color": מערכים אשר ייצגו נקודות ואוחסנו ב-DS, ובעזרת פעולה, צוירו על המסך (כמו מערכים שונים שמייצגים "תמונות" בגדלים שונים של האויב). עבור מערכים אלו, לא פעם יש צורך בהחלפה למערך המייצג את אותה דמות במרחק אחר (קטנה / גדולה יותר) ולכן קיימות פרוצדורות אשר מסנכרנות את המערכים כך שברגע שמוחלף מערך, כולם נמצאים באותו מקום.
 3. אובייקטים מורכבי קווים: קיימים אובייקטים רבים המורכבים מהנקודות הקיצוניות ביותר בקו ומקואורדינטת Z משותפת. לדוגמה, קו ים המצויר מנקודה הנמצאת בקצה קרוב לסוף הים מצד שמאל, ומנקודה הנמצאת בקצה קרוב לסוף הים מצד ימין. לאחר מכן, משורטט הקו, ומקטינים את ערך ה-Z (המייצג מרחק) כך שנראה שהגלים מתקרבים³.
 4. נקודות דו ממדיות פשוטות: קיימות גם נקודות דו ממדיות פשוטות, לרוב מופיעות להצגת מידע בפשטות (מהירות, גובה).
- העולם עצמו מוצג כעולם תלת ממדי, שכולו מסתובב סביב השחקן. האובייקטים זזים לפי תנועת השחקן, ובסיבוב מלא יופיעו שוב (אם טרם נעלמו במקרה של אויבים או טילים).
- עבור אובייקטים תלת ממדיים, קיימת פרוצדורה המחשבת את מיקומם במסך הדו ממדי על פי מיקומם בעולם התלת ממדי (על ידי לקיחת הפרספקטיבה – המרחק ושדה הראייה המוגדר וחישובם יחד עם נקודות X, Y). לאחר מכן, מצויר קו דרך אלגוריתם המוצא את הדרך הכי קצרה בין שתי נקודות. עבור הוספת "רעש" לגלים ככה שנראים זזים, הוספתי משתנה שבאופן רנדומלי עלולה להזיז את נקודת ה-Y בעת ציור קו ל--y. עם זאת, הקליעים לדוגמה, הינם קווים אחידים.
- עם שינוי קואורדינטת Z, נראה שהאובייקט מתקרב או מתרחק. במשחק, באופן תמידי מגדילים את קואורדינטת ה-Z של הגלים כך שהם נדמים מתקרבים, עד שמאפסים אותם חזרה לקו האופק עם Z מוגדר. עם הגברת / הפחתת המהירות, קצב שינוי ה-Z משתנה.
- יש לשים לב גם לניהול הצלילים והמוזיקה:
1. צלילים פשוטים: ישנם צלילים פשוטים, דוגמת צלילי היריות. הפרקטיקה היא שליחת תדר, והעלאת / הורדת התדר ליצירת סאונד למשך זמן מוגדר.
 2. מוזיקת רקע: ישנה מוזיקת רקע המורכבת מתדרים ומספר טיקים מוגדרים המנוגנת כל איטרציה לולאה.
 3. שירים: ישנם שירים המתנגנים במסך הפתיחה ובמסך הראשי (וגם קטעי שמע ב-CUTSCENE). ניתן לראות את אופן ניהול תהליך זה בתרשים הזרימה. שירים מנוגנים על ידי פעולה ייעודית, כאשר המידע עבור השיר נקרא מתוך קבוצת RAW.

³ עוד על שיטה זו כתוב בהמשך.

בנוסף לעיל, להלן תיאור האלגוריתם המרכזי (קיים גם תרשים זרימה):

איפוס:

איפוס משתני אויבים, זימון אויבים, איפוס מיקום טיל, סנכרון כל הטילים.

מסך פתיחה:

ציור מסך פתיחה.
השמעת שיר פתיחה.
המתנה עבור לחיצה.
אם לא נלחץ כלום, המשך ניגון השיר. אחרת:
הפעלת מקלדת אסינכרונית.

מסך ראשי:

סימון את מקש ESC כלא לחוץ.
ביטול מקלדת אסינכרונית.
ציור מסך ראשי.
השמעת שיר מסך ראשי.
המתנה עבור לחיצה / סוף השיר.
ניגון CUTSCENE. אם נוגן, דלג.
הפעלת מקלדת אסינכרונית.

לולאה ראשית:

אם נלחץ ESC, חזור למסך ראשי.
אם השחקן נפסל, סיים את המשחק.
מלא את רקע העולם.
השמע טיק מוזיקת רקע.
בדוק לחיצות על מקשים והזז את השחקן.
זוז קדימה.
צייר גלים.
נהל מערכות אויבים.
נהל מערכות נשקים.
נהל טילי אויבים.
צייר את מטוס השחקן.
נהל גובה.
נהל מהירות.
צייר הכל למסך.
חזור ל לולאה ראשית.

סיים את המשחק:

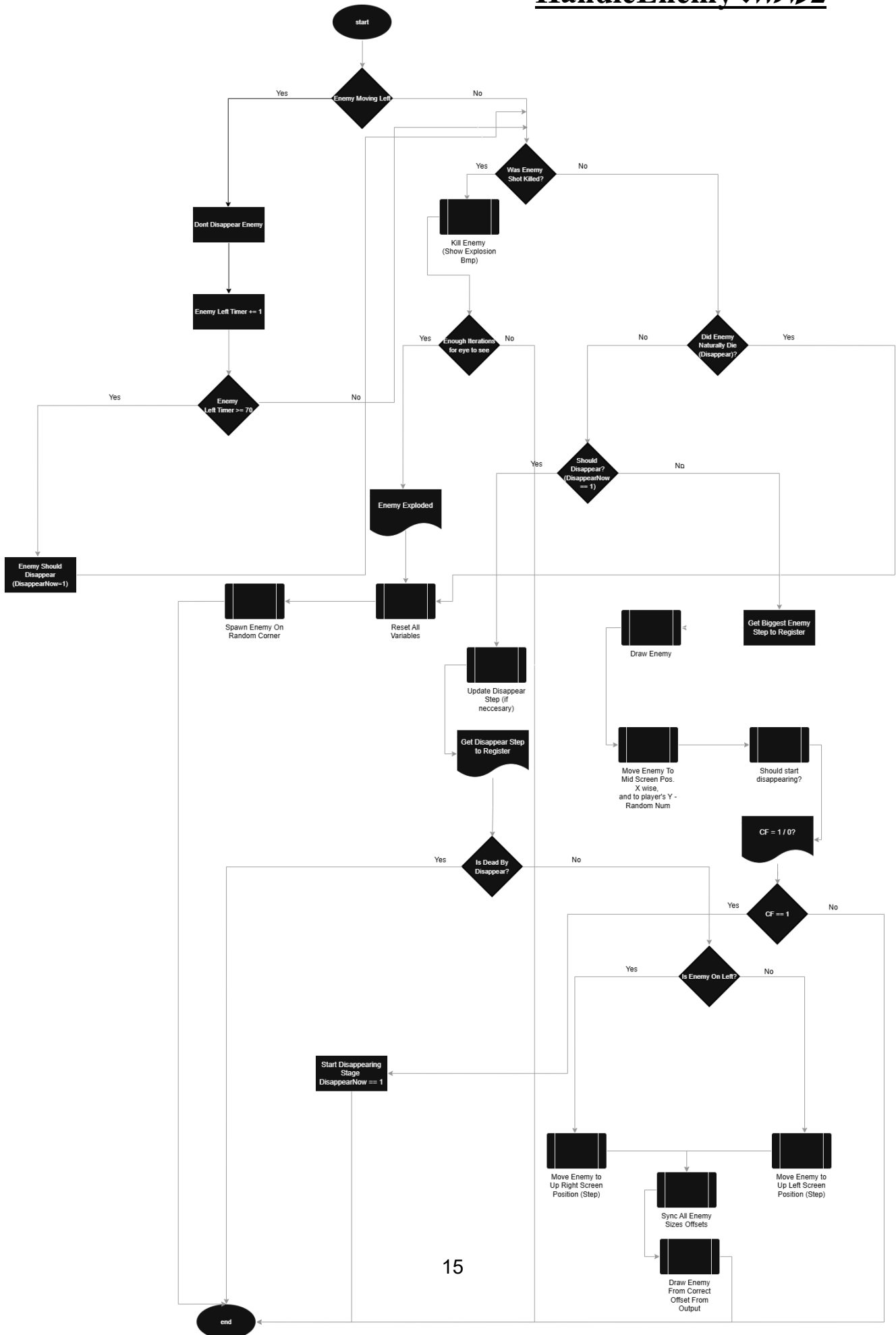
כבה את הרמקול.
חזור למצב טקסט.
צא מהתוכנית.

תרשימי זרימה

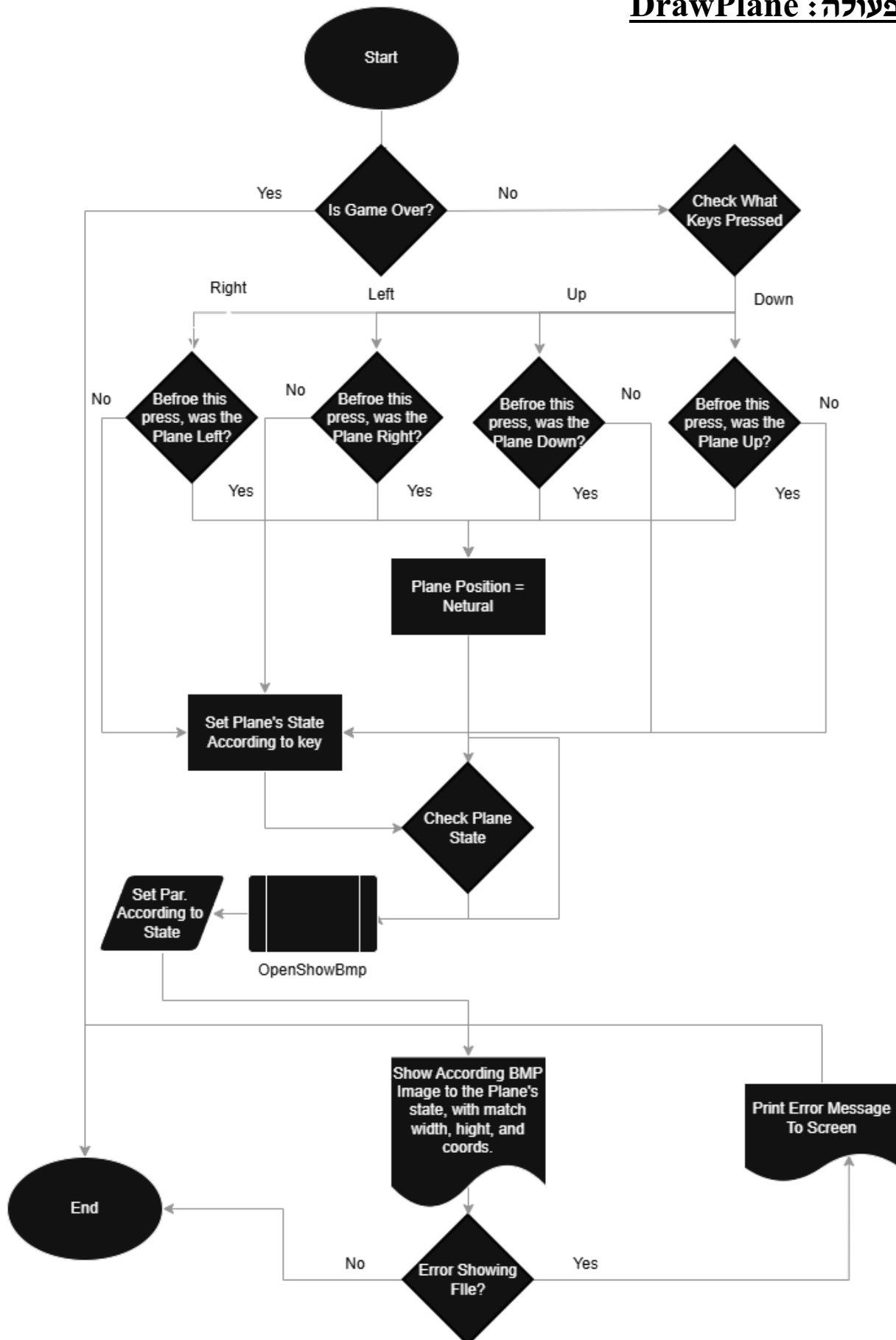
הפעולות המרכזיות להן יוצגו תרשימי זרימה הן:

1. **HandleEnemy** (ניהול אויב).
2. **DrawPlane** (ניהול מטוס שחקן).
3. **TranistionBuffer** (כתיבה למסך).
4. **HandleStartCutscene** (ניגון סיפור רקע למשחק).
5. **HandleShooting** (ירי רובי מטוס השחקן).
6. **LaunchMissile** (שיגור טיל אל עבר השחקן).
7. **DrawFromPixelFormatArray** (ציור מערך פיקסלים).
8. **OpenShowBmp** (פתיחת וציור תמונת BMP 256 Bitmap).
9. **PlaySong** (ניגון שירים שהומרו לקובץ RAW בעזרת Audicity).
10. **MainGameLoop** (לולאת המשחק הראשית, אינה פעולה).

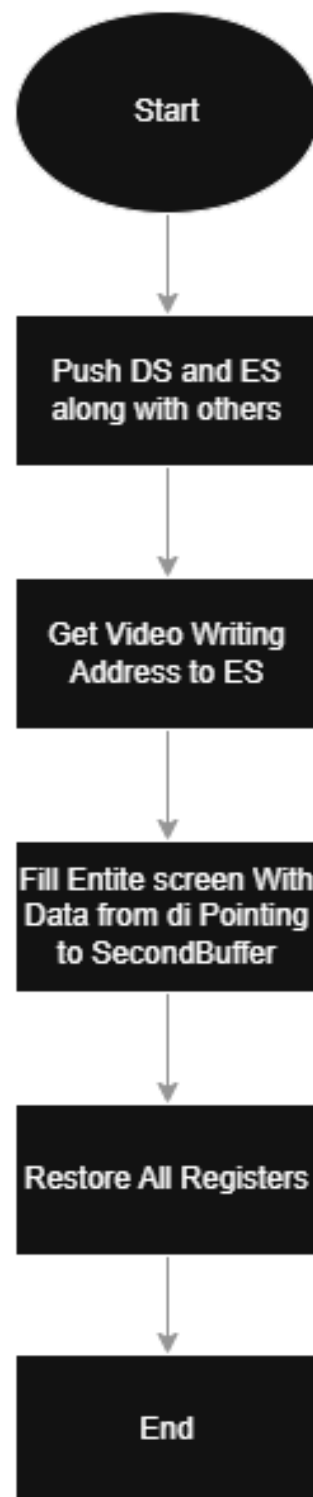
פעולה: HandleEnemy



פעולה : DrawPlane

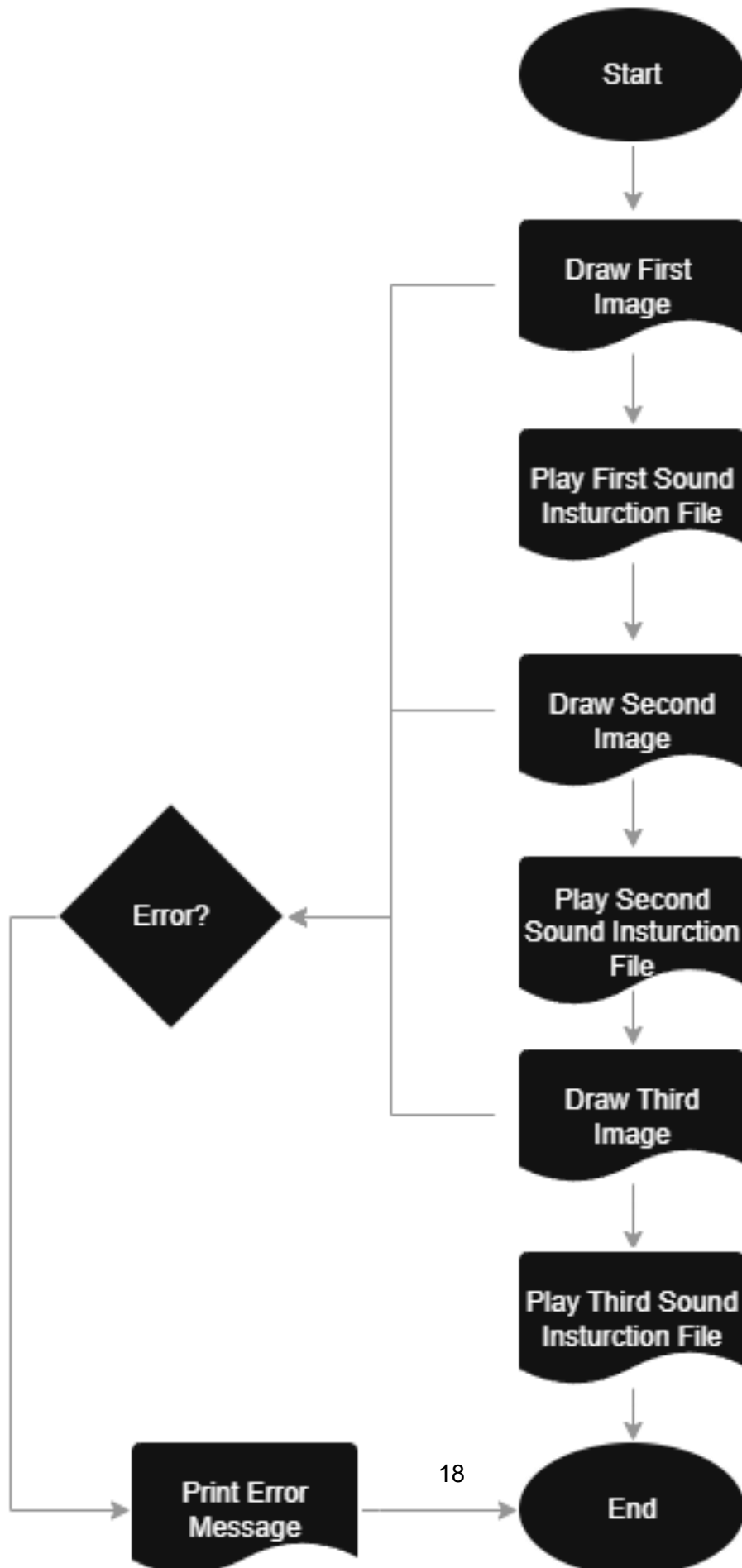


פעולה : TranistionBuffer

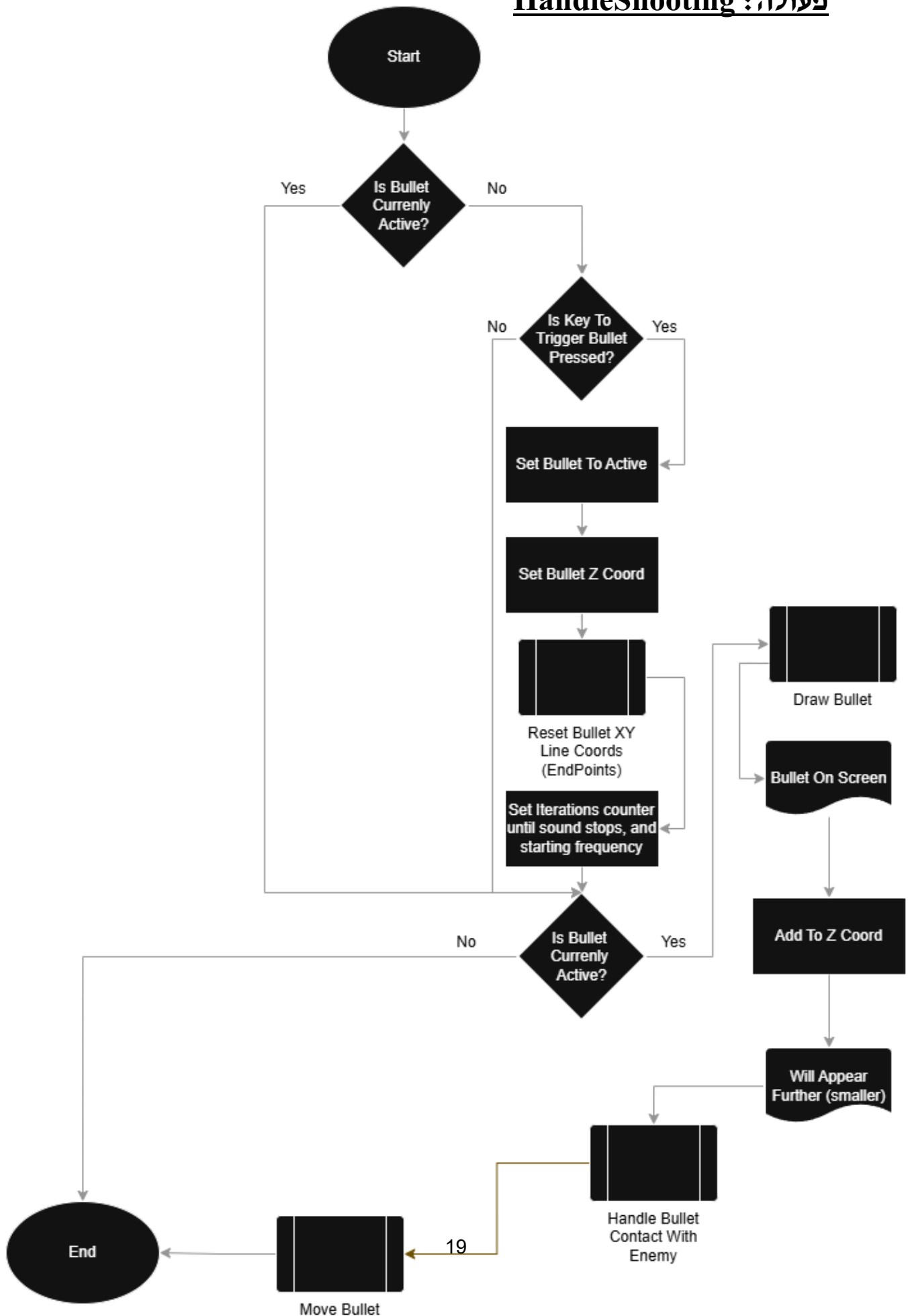


פעולה: PlayStartCutscene

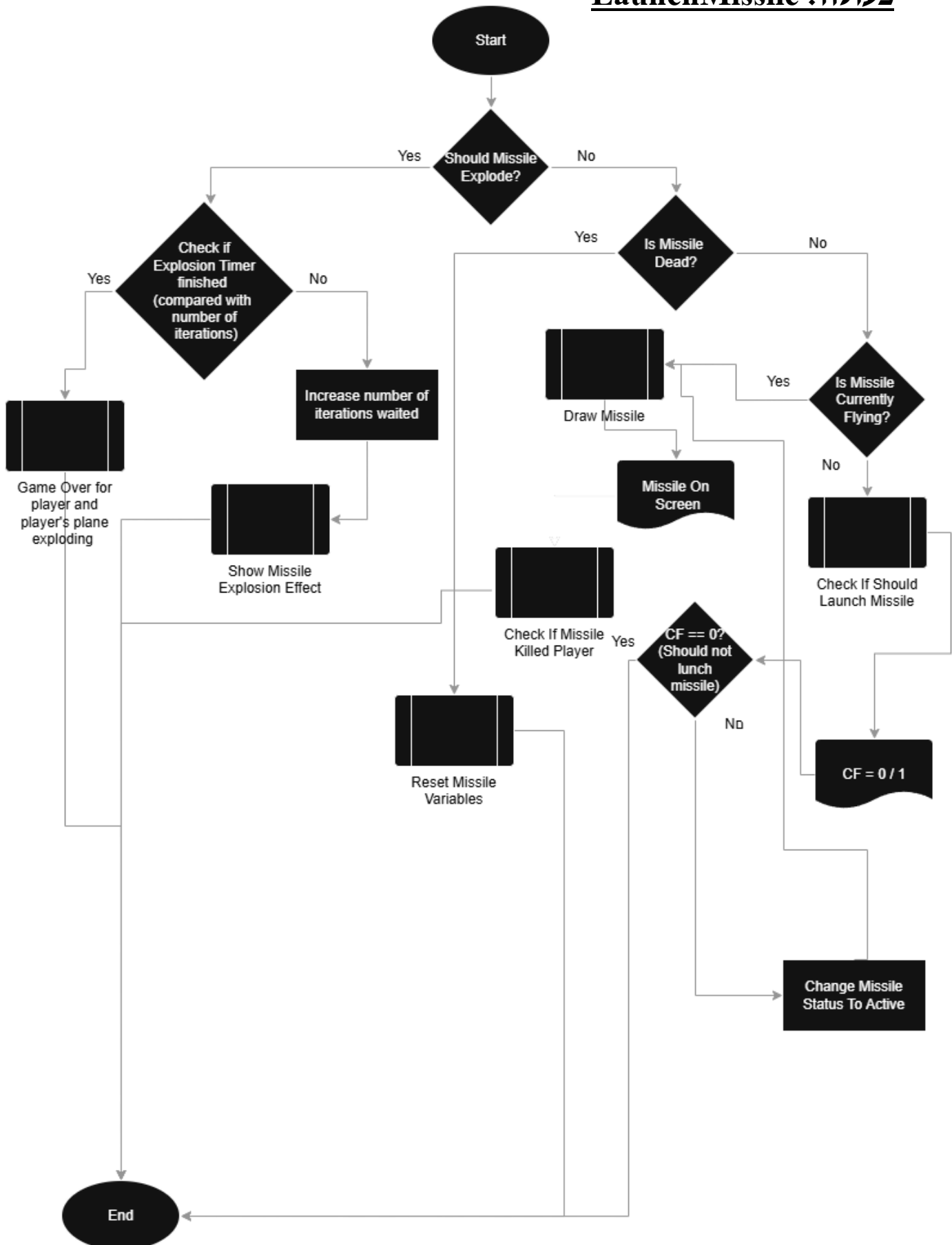
הערה: קודם לפעולות של הצגת תמונה ושמייעת ההוראות נקראות פעולות. עבור הפשטה, מוצג Input בלבד. אותה פעולה נקראת בהתאם למצב (קיימת פעולה להצגת תמונה ופעולה לניגון שמע).



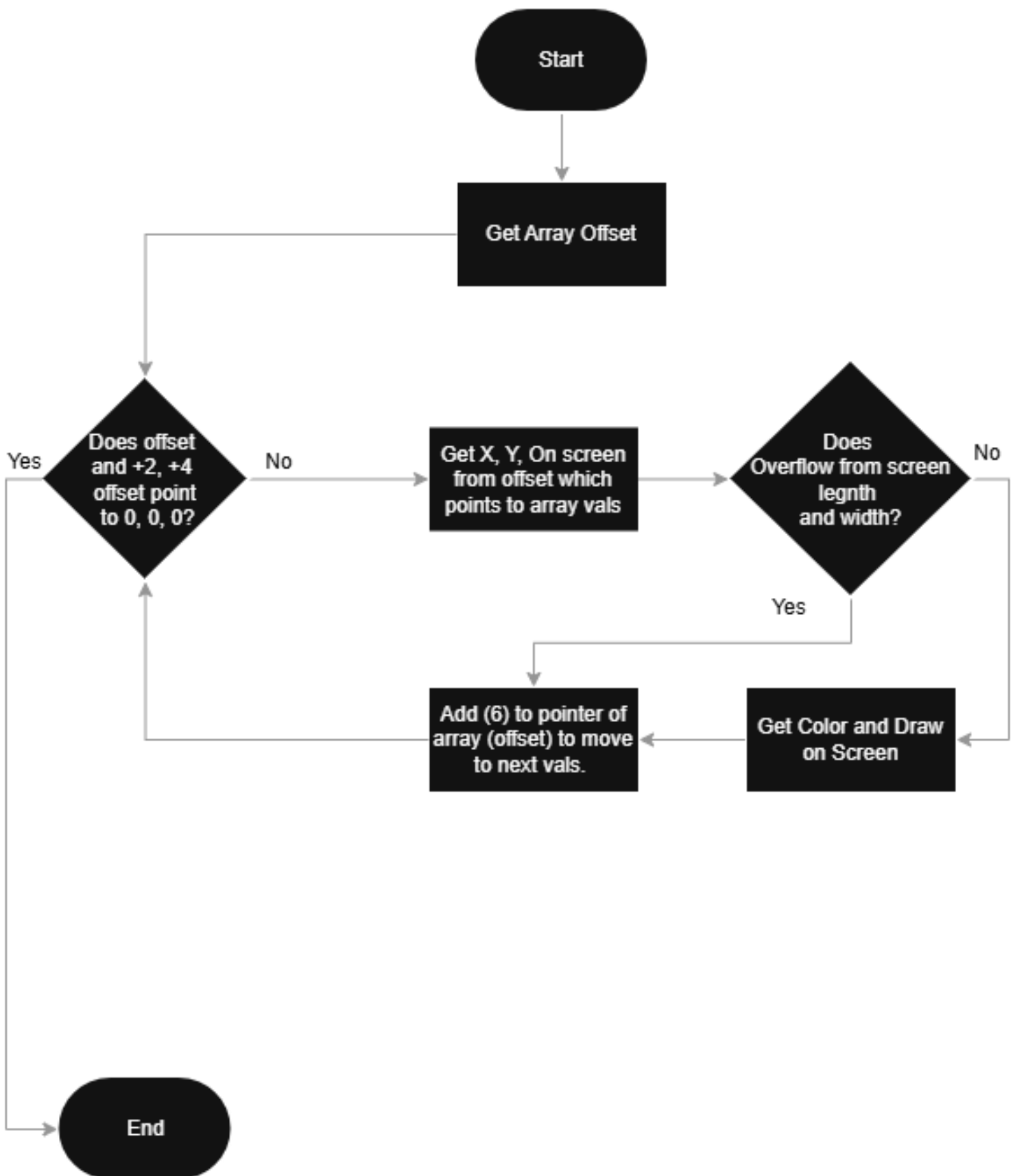
פעולה: HandleShooting

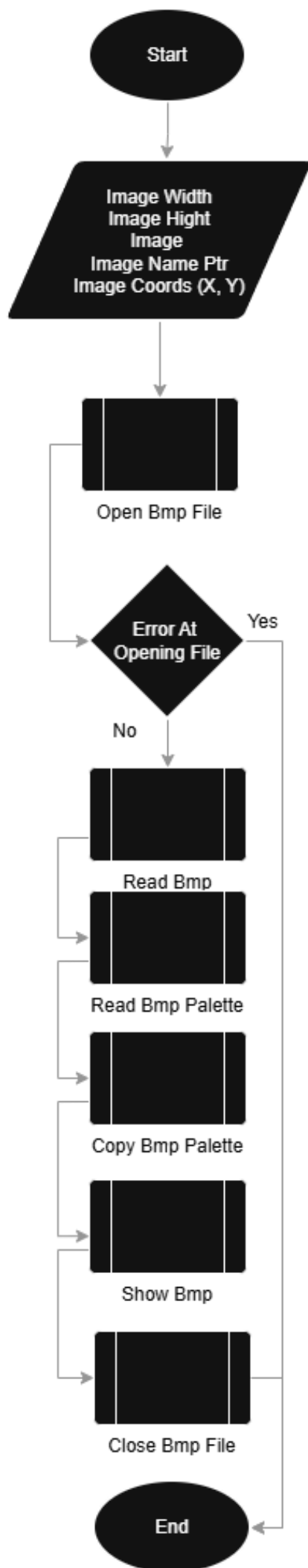


פעולה : LaunchMissile



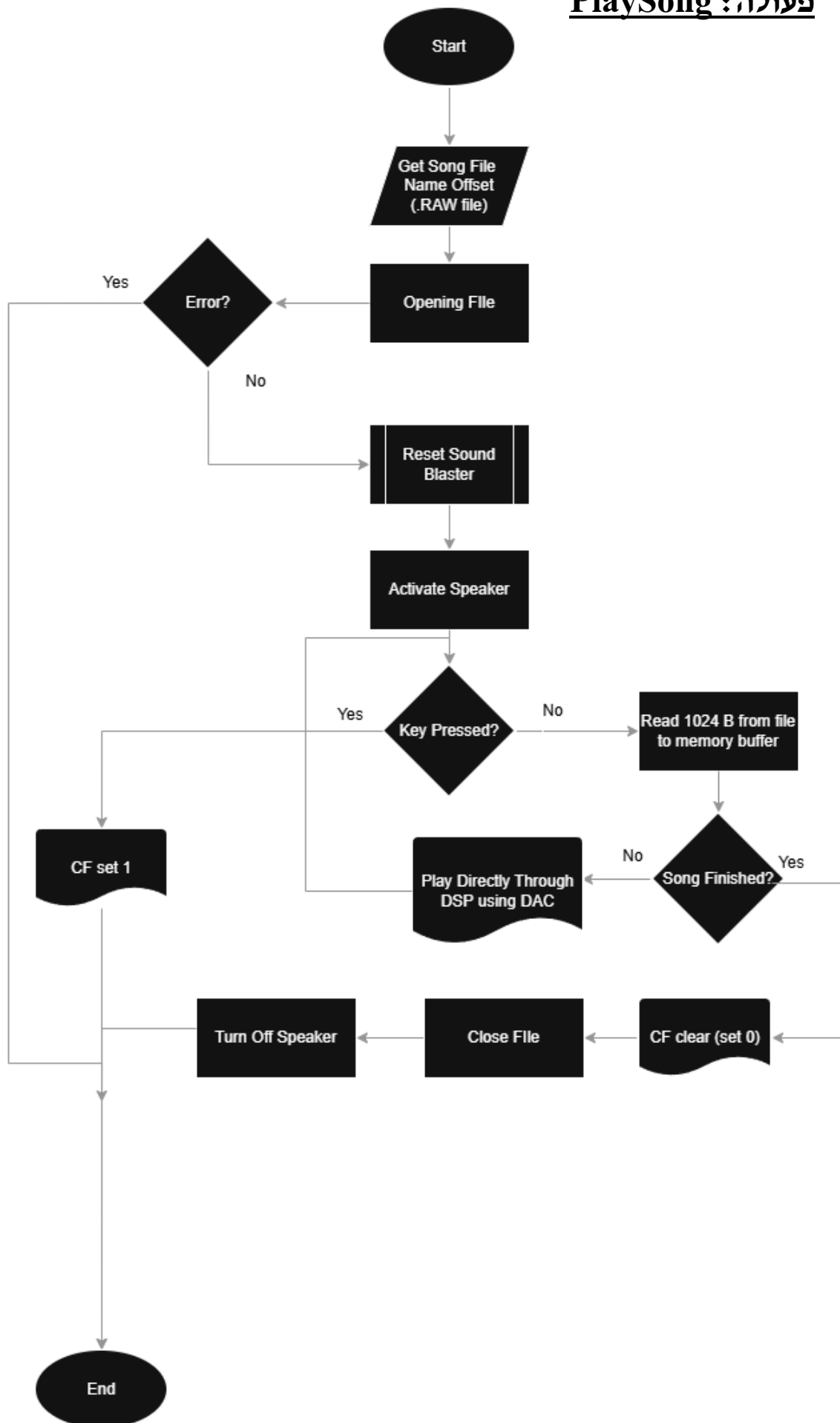
פעולה : DrawFromPixelFormat



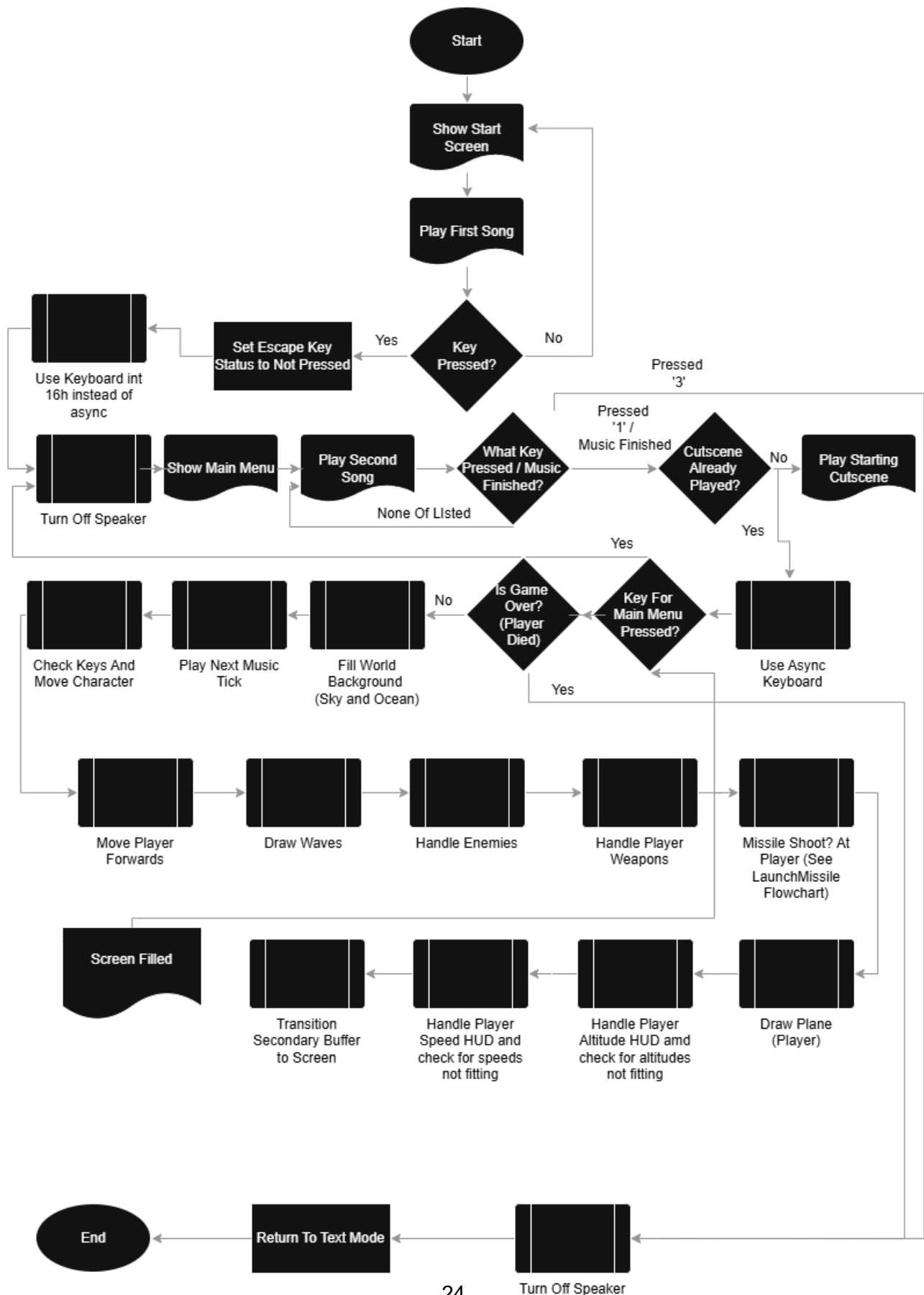


פעולה : OpenShowBmp

פעולה : PlaySong



לולאה ראשית: MainGameLoop



רשימת הפעולות

```
;
=====
=====
; PROC TickMusic
; -----
; Purpose : Advance the background music by one tick.
;     Checks whether the current note has played long enough; if so,
;     moves to the next note and calls PlayTone.
;
; Entry  : music_data  - array of (frequency, duration) word pairs
;     current_note_index - index of the currently playing note
;     note_timer  - frames elapsed since current note started
;
; Exit   : current_note_index and note_timer updated.
;     PlayTone called when a note transition occurs.
;
; Modifies: (none - all registers preserved via push/pop)
;
=====
=====
----
;
=====
=====
; PROC TickMusic
; -----
; Purpose : Advance the background music by one tick.
;     Checks whether the current note has played long enough; if so,
;     moves to the next note and calls PlayTone.
;
; Entry  : music_data  - array of (frequency, duration) word pairs
;     current_note_index - index of the currently playing note
;     note_timer  - frames elapsed since current note started
;
; Exit   : current_note_index and note_timer updated.
```

```

;      PlayTone called when a note transition occurs.
;
; Modifies: (none - all registers preserved via push/pop)
;
=====
=====
----
;
=====
=====
; PROC CopyPixelArray
; -----
; Purpose : Copy a pixel-triplet array (X, Y, Color word triplets terminated
;          by 0,0,0) from one array to another.
;
; Entry  : [bp+4] = source array offset
;          [bp+6] = destination array offset
;
; Exit   : Destination array is an exact copy of source.
;
; Modifies: Destination array contents only.
;
=====
=====
----
;
=====
=====
; PROC Add_XY_ToMissile
; -----
; Purpose : Apply a fixed initial (X, Y) offset to every entry in the
;          Missile_XXXS array so that the missile first appears at a
;          reasonable starting position on screen.
;
; Entry  : Missile_XXXS must contain freshly initialised relative coordinates.
;
; Exit   : All X fields in Missile_XXXS incremented by 70.
;          All Y fields in Missile_XXXS incremented by 50.
;
; Registers Effected: None.

```

```

;
=====
=====
----
;
=====
=====
; PROC LaunchMissile
; -----
; Purpose : Decides whether to wait for the spawn timer, update the active
;         missile, handle its explosion, or reset it after death.
;
; Entry   : MissileExploded, killMissile, isMissileActive - current state flags.
;
; Exit    : Missile drawn/updated, explosion shown, Game Over, or state reset.
;
=====
=====
----
;
=====
=====
; PROC CheckMissileHitPlayer
; -----
; Purpose : Scan every pixel in the Missile_M sprite (the medium size)
;         to see whether it overlaps the player's aircraft bounding box.
;         Only the M-stage sprite is tested because earlier stages are too
;         far away to plausibly hit the player, and close ones are triggered
;         too late.
;
; Entry   : GetMissileSizeToSi must succeed and return SI = Missile_M.
;         Player aircraft is assumed to occupy pixels X: 135-198, Y: 88-136.
;         (BmpLeft=115, sprite width=83. right edge=198;
;         BmpTop=88, sprite height=48. bottom edge=136.
;         The X left bound is tightened to harden hits for the missile.
;
; Exit    : If a pixel overlaps the player box:
;         isMissileActive = 0, killMissile = 1, MissileExploded = 1.

;         No registers modified (all saved/restored).

```

```

;
=====
=====
----
;
=====
=====
; PROC ExplodeMissile
; -----
; Purpose : Render a two-stage missile explosion animation at the missile's
;         current screen position.
;
; Entry  : Missile_XXXS[0] = current X, Missile_XXXS[2] = current Y
;         ExplodeStage2Timer = frame count within the explosion sequence.
;
; Exit   : BMP drawn to the buffer.
;         ExplodeStage2Timer incremented.
;         Following will be another BMP, to create a cool explosion effect.
;
; Modifies: ExplodeStage2Timer, FileError, BmpLeft, BmpTop, BmpWidth,
BmpHeight, FileNamePtr
;
=====
=====
----
;
=====
=====
; PROC CheckIfShouldLaunchMissileOnPlayer
; -----
; Purpose : The missile shooting time logic. When it reaches the current
;         limit, the proc sets carry (signalling the caller to fire) and
;         picks a new random limit.
;
; Entry  : enemyShootOnPlayerCountdown - frames elapsed since last shot.
;         enemyShootOnPlayerCountdownLimit - target frame count before firing.
;
; Exit   : Carry set = fire now (countdown reset, new random limit chosen).
;         Carry clear = not yet (countdown incremented).
;

```

```

;
=====
=====
----
;
=====
=====
; PROC DrawMissileNew
; -----
; Purpose : Each frame, determine the current missile size stage, move the
;           Missile_XXS coordinates in the appropriate direction,
;           synchronise all size sprites, and render the active size sprite to the
;           off screen buffer.
;
; Entry  : SI = size sprite pointer (set by GetMissileSizeToSi).
;           cxMODEMissile / CxSizeTimerMissile control speed of size growth.
;
; Exit   : Missile_XXS coordinates updated.
;           Active size sprite rendered.
;
;
=====
=====
----
;
=====
=====
; PROC setKeyboardInterrupt
; -----
; Purpose : install KeyboardInterrupt as the active handler for the replaced int.
;
; Entry  : keyboardInterruptPOS - constant. the byte offset of the INT 09h entry.
;           KeyboardInterrupt - offset of the custom int services routine to run.
;
; Exit   : currentInterruptPOS = keyboardInterruptPOS (36).
;           currentInterruptOFFSET = offset of KeyboardInterrupt.
;           IVT INT 09h entry patched to point at KeyboardInterrupt in the current
code segment (via SetInterrupt).
;           OldKeyboardInterruptOffset = original INT 09h handler offset.
;           OldKeyboardInterruptSegment = original INT 09h handler segment.
;

```

```

;
=====
=====
----
;
=====
=====
; PROC restoreKeyboradInterrupt
; -----
; Purpose : Undo the keyboard ISR installation performed by
;         setKeyboradInterrupt. Copies the saved original handler address
;         from OldKeyboardInterruptOffset / OldKeyboardInterruptSegment back.
;
; Entry  : OldKeyboardInterruptOffset - original INT 09h handler offset
;         saved by setKeyboradInterrupt.
;         OldKeyboardInterruptSegment - original INT 09h handler segment.
;         keyboardInterruptPOS      - constant 36 (9*4); IVT byte offset
;         for INT 09h.
;
; Exit   : CurrentOldInterruptOffset = OldKeyboardInterruptOffset.
;         CurrentOldInterruptSegment = OldKeyboardInterruptSegment.
;         INT 09h entry restored to the original BIOS handler
;         (via RestoreOldInterrupt).
;
;
=====
=====
----
;
=====
=====
; PROC SetInterrupt
; -----
; Purpose : Install a new interrupt handler by patching the interrupt vector
;         table (at es:0) at the position specified by currentInterruptPOS.
;
; Entry  : currentInterruptPOS - byte holding the IVT byte-offset of the
;         interrupt to replace.
;         currentInterruptOFFSET - word holding the offset of the new
;         handler routine to install.
;

```

```

; Exit : IVT entry at currentInterruptPOS patched with the new handler.
; CurrentOldInterruptOffset = old int offset (saved).
; CurrentOldInterruptSegment = old int segment (saved).
;
; All registers preserved (PUSHA / POPA).
;
;
=====
=====
----
;
=====
=====
; PROC RestoreOldInterrupt
; -----
; Purpose : Undo the effect of SetInterrupt by writing the previously saved
; handler (offset + segment) back into the IVT at the position
; specified by currentInterruptPOS, restoring the original ISR.
;
; Entry : CurrentOldInterruptOffset - saved offset of the original handler.
; CurrentOldInterruptSegment - saved segment of the original handler.
; currentInterruptPOS - byte holding the IVT byte-offset of
; the interrupt to restore.
;
; Exit : IVT entry at currentInterruptPOS restored to the original handler.
; All registers preserved (PUSHA / POPA).
;
;
=====
=====
----
;
=====
=====
; PROC KeyboardInterrupt (FAR)
; -----
; Purpose : Custom hardware interrupt service routine keyboard.
; Reads the scan code from the keyboard controller, strips the release bit to
; obtain the key index, and records the key state (pressed or released) in the
; keys[] array. Async.
;

```

```

; Entry : keys[]      - 128-byte array indexed by scan code.
;      Port 60h      - keyboard data port; yields the raw scan code.
;
; Exit  : keys[scanCode] = 1 on key press, 0 on key release.
;      anyKeyPressed  updated by AnyKeyPressedCheck.
;      All registers preserved (PUSHA / POPA + DS).
;
;
=====
=====
----
;
=====
=====
; PROC AnyKeyPressedCheck
; -----
; Purpose : Scan the entire keys[] array (128 bytes) to determine whether
;      any key is currently held. Sets anyKeyPressed = 1 if found,
;      or 0 if no key is pressed.
;
; Entry : keys[] array reflects the current keyboard state.
;
; Exit  : anyKeyPressed = 0 or 1.
;
; Modifies: None.
;
=====
=====
----
;
=====
=====
; PROC SyncAllMissiles
; -----
; Purpose : After moving Missile_XXXS, sync the same relative position delta to
;      all larger missile sprites (XXS, XS, S, M, L) so that every size
;      variant renders at the same screen location.
;
; Entry : Missile_XXXS contains the current (moved) position.
;      All other sprites contain their last-synced position.
;

```



```

; Exit : All missile sprites have field[0] (X) and field[2] (Y) equal to
;       those in Missile_XXS in coordination to their size (of course it can not be
;       the exact same coords).
;
; Modifies: All missile sprite (except ready XXS) array's X and Y fields.
;
=====
=====
----
;
=====
=====
; PROC GetMissileSizeToSi
; -----
; Purpose : Get the current missile sprite size according to time passing and load to SI
;          - to serve as the pointer to the sprite.
;
; Entry : cxMODEMissile / CxSizeTimerMissile (consumed by GetCX), Missile Sprites
;        (from inside the proc).
;
; Exit : SI = offset of the active sprite.
;       Carry clear = valid sprite returned.
;       Carry set = missile exceeded maximum range; killMissile set to 1.
;
;
=====
=====
----
;
=====
=====
; PROC GetCX
; -----
; Purpose : Advance the missile size counter (cxMODEMissile) using a
;          shift-register "timer", then compute CX = cxMODEMissile * 100.
;
; The shift-register "timer" (CxSizeTimerMissile) slows down the size
; progression
;
; Entry : cxMODEMissile, CxSizeTimerMissile.
;

```

```

; Exit  : CX = cxMODEMissile * 100.
;        cxMODEMissile possibly incremented (max 40).
;
; Modifies: CX
;
=====
=====
----
;
=====
=====
; PROC FreqToDivisor
; -----
; Purpose : Convert a frequency in Hz to the PIT (8253) channel 2 divisor
;           so the correct tone is produced by the PC Speaker.
;
; Entry  : AX = desired frequency in Hz (must be > 0).
;
; Exit   : AX = PIT divisor to send to port 42h.
;
; Modifies: AX (result)
;
=====
=====
----
;
=====
=====
; PROC PlayTone
; -----
; Purpose : Emit a tone at the requested frequency.
;
; Entry  : AX = frequency in Hz
;
; Exit   : PC Speaker emitting the requested tone.
;
; Modifies: None
;
=====
=====
----

```

```

;
=====
=====
; PROC FindColorUnderMouse
; -----
; Purpose : check the mouse for a left-button click, read the screen
;          pixel color at the cursor position, and display the color index
;          as a decimal number on the text console.
;          Useful only during development/debugging; not called in the
;          final game loop.
;
; Entry   : Graphic mode must be set.
;
; Exit    : If left button was pressed:
;          - Mouse cursor hidden while reading.
;          - Colour index at cursor position printed via ShowAxDecimal.
;          - Mouse cursor restored.
;          All registers preserved.
;
;
=====
=====
----
;
=====
=====
; PROC debugFindColors
; -----
; Purpose : Fill secondBuffer with a rising colour-index ramp
;          (0, 1, 2, ... 255, 0, 1, ...) so that every VGA palette entry is
;          visible on screen when the buffer is blitted.
;          Debug/development utility only – not called in the final game.
;
; Entry   : secondBuffer segment must be defined.
;
; Exit    : secondBuffer[0..63999] filled with bytes 0-255 repeating.
;          All registers preserved.
;
; Arbitrary numbers explained:
; 64000 - total bytes in a 320×200 VGA Mode-13h frame (320 * 200 = 64 000).

```

```

;
=====
=====
----
;
=====
=====
; PROC HandleWeapons
; -----
; Purpose : Top-level dispatcher for all player weapon systems.
;         Currently delegates entirely to HandleGuns.
;         Designed as an extension point so future weapon types
;         (missiles, bombs, etc.) can be added here without touching
;         the main game loop.
;
; Entry  : None.
; Exit   : None (all state changes occur inside called procedures).
; Modifies: What HandleGuns modifies (see that proc).
;
=====
=====
----
;
=====
=====
; PROC HandleGuns
; -----
; Purpose : Each-frame update for the player's gun system.
;         Runs three sub-systems in order:
;         1. HandleShooting – fire logic and bullet physics.
;         2. HandleShootSound – PC-speaker audio for the gun.
;         3. loadGunsCrosshair – draw the HUD aiming reticle.
;
; Entry  : None.
; Exit   : Shooting control ON-SCREEN.
; Modifies: What the three called procedures modify.
;
=====
=====
----

```

```

;
=====
=====
; PROC HandleShootSound
; -----
; Purpose : Drive the PC-speaker shooting sound effect one frame at a time.
;
; Entry : ShootSoundTimer – frames remaining in the sound effect (0 = end).
;       ShootSoundFreq – current speaker frequency in Hz.
;
; Exit : ShootSoundTimer decremented by 1 (if > 0).
;       ShootSoundFreq incremented by 200 (if timer was > 0).
;
;
=====
=====
----
;
=====
=====
; PROC HandleShooting
; -----
; Purpose : Each-frame update for the player bullet. Spawn it, Move it, Handle sound
variables,
;       check hits, etc...
;
; Entry : BulletActive – 0 = idle, 1 = in flight.
;       keys[] – keyboard state array (1 = held).
;       BulletLines_generalZ – current depth of the bullet box.
;
; Exit : BulletActive may be set to 1 (fire) or 0 (expired/hit).
;       BulletLines_generalZ incremented by 10 each active frame.
;       ShootSoundTimer / ShootSoundFreq initialised on new shot.
;
=====
=====
----
;
=====
=====
; PROC ResetBulletCoords

```

```

; -----
; Purpose : Restore every bullet-line endpoint to its default centred
;           position so that a newly fired bullet starts as two small
;           squares (left gun + right gun) centred on the player's 'guns'.
;
; Entry   : None.
; Exit    : BulletLine1-4 (left box) and BulletLine1R-4R (right box)
;           endpoint variables reset to default values.
;
;
=====
=====
----
;
=====
=====
; PROC KillEnemy
; -----
; Purpose : Animate the death of Enemy 1 in two stages:
;           Stage 1 – show a small explosion BMP (E_ES.bmp) at the enemy
;           position for 5 frames.
;           Stage 2 – show a large explosion BMP (E_EB.bmp); move the enemy
;           sprite downward 6 pixels per frame simulating it
;           falling, until Y > 170 at which point carry is set
;           to signal the caller that the kill sequence is done.
;
; Entry   : EnemyX / EnemyY – screen position of the dead enemy.
;           StageOneEnemyExplosionComplete – 0 = still in stage 1, 1 = stage 2.
;           WaitForEnemyExp2 – frame counter for stage 1.
;
; Exit    : Carry set = explosion finished (enemy fell off screen).
;           Carry clear = explosion still running.
;           EnemyY incremented by 6 each stage-2 frame.
;           StageOneEnemyExplosionComplete set to 1 after 5 stage-1 frames.
;
;
=====
=====
----

```

```

;
=====
=====
; PROC CheckBulletCoords
; -----
; Purpose : Test the current bullet bounding box against all active, living
;          enemies (1, 2, and 3). On a confirmed hit the bullet is consumed
;          and the appropriate "dead by fire" flag is set.
;
; Entry   : BulletActive, Enemy2Active, Enemy3Active, isDead2, isDead3 –
;          BulletLines_generalZ – bullet depth (hit only counts if ≥ 200).
;          EnemyNormal / Enemy2Normal / Enemy3Normal – pixel arrays.
;
; Exit    : On hit AND Z ≥ 200:
;          BulletActive = 0.
;          EnemyDeadByFireN = 1 for the struck enemy.
;          EnemyXN / EnemyYN = struck enemy's current first-pixel coords.
;
; Arbitrary numbers explained:
; 200 – minimum depth for a hit to register.
;
=====
=====
----
;
=====
=====
; PROC CheckBulletCoordsWithEnemySI
; -----
; Purpose : Test all 8 projected bullet corners against the enemy pixel array
;          pointed to by SI. Returns carry set if any corner matches.
;
; Entry   : SI = offset of the enemy pixel array (word triplets X,Y,Color).
;          BulletScreenX1-X8, BulletScreenY1-Y8 – projected corner coords.
;
; Exit    : Carry set = at least one corner hit a pixel in the enemy array.
;          Carry clear = no hit.
;          AX, BX, CX preserved.
;
=====
=====

```

```

----
;
=====

=====
; PROC CheckBulletCornerSI
; -----
; Purpose : Check the enemy pixel array pointed to by SI and return carry set
;           if the screen coordinate (AX, BX) matches any pixel entry.
;           Termination sentinel: three consecutive zero words (0,0,0).
;
; Entry  : SI = enemy pixel array pointer (word triplets: X, Y, Color).
;           AX = bullet screen X to test.
;           BX = bullet screen Y to test.
;
; Exit   : Carry set = (AX, BX) found in the array.
;           Carry clear = (AX, BX) not found (end of array reached).
;           SI, AX, BX preserved.
;
=====

=====
----
;
=====

=====
; PROC GetBulletScreenCoords
; -----
; Purpose : Project all 8 world-space corners of the bullet's two 3-D boxes
;           onto the 2-D screen using Calc3D (perspective projection) and
;           store the results in BulletScreenX1-X8 / BulletScreenY1-Y8.
;           Must be called once per frame before any bullet hit-testing.
;
; Entry  : BulletLine1/2/3/4 _X0,_X1,_Y0,_Y1 – left box endpoints.
;           BulletLine1R/2R/3R/4R _X0,_X1,_Y0,_Y1 – right box endpoints.
;           BulletLines_generalZ – shared Z depth for both boxes.
;
; Exit   : BulletScreenX1-X8 / BulletScreenY1-Y8 updated with projected coords.
;           AX, BX, CX preserved.
;
;
=====

=====

```



```

----
;
=====
=====
; PROC HandleBulletMovement
; -----
; Purpose : Each frame, decide how the bullet box should move
;
; Entry  : BulletActive      – 0 = idle, 1 = in flight.
;         BulletLines_generalZ – current Z depth.
;
; Exit   : BulletActive set to 0 if Z ≥ 300.
;         Box coordinates updated via AddXYToBulletsLines / SmallerInPlace.
;
;
=====
=====
----
;
=====
=====
; PROC SmallerInPlace
; -----
; Purpose : Advance the bullet's Z depth by 15 units (faster than normal)
;         and nudge both boxes one pixel inward/upward, creating the visual
;         impression that the bullet is shrinking as it recedes.
;         Called each frame during a shrink phase.
;
; Entry  : BulletLines_generalZ
; Exit   : BulletLines_generalZ += 15. All box endpoints shifted by (-1,-1).
;
;
=====
=====
----
;
=====
=====
; PROC AddXYToBulletsLines
; -----
; Purpose : Add a x/y offset to all 8 line endpoints of both

```

```

;      the left and right bullet boxes.
;
; Entry  : [bp+4] = deltaX
;      [bp+6] = deltaY
;
; Exit   : BulletLine X and Y fields updated.
;      AX preserved.
;
;
=====
=====
----
;
=====
=====
; PROC HandleShootingGraphics
; -----
; Purpose : Render the player's bullet each frame as two small 3-D boxes
;      (left gun + right gun) by projecting each line endpoint with
;      Calc3D, then drawing the resulting screen-space lines via
;      Bresenham_GetPoints + DrawPoints.
;
; Entry   : BulletLine1-4 (left box) and BulletLine1R-4R (right box) endpoint
;      variables must contain current world-space coordinates.
;      BulletLines_generalZ – current Z depth shared by both boxes.
;
; Exit    : Lines drawn to secondBuffer.
;      AX, BX, CX, SI, DI preserved.
;
;
=====
=====
----
;

```

```

;
=====
=====
; PROC loadGunsCrosshair
; -----
; Purpose : Draw the player's gun aiming reticle (crosshair) onto the
;           secondary buffer each frame by rendering the GunsCrosshair
;           pixel-triplet array via DrawFromPixelArray.
;
; Entry   : GunsCrosshair - pixel-triplet array (X, Y, Color) defining the
;           crosshair shape at its fixed screen position.
;
; Exit    : GunsCrosshair pixels written to secondBuffer.
;
;
=====
=====
;
=====
=====
; PROC playStartCutscene
; -----
; Purpose : Play the one-time intro cutscene shown before the first game
;           session. Displays three full-screen story BMP images in sequence,
;           each accompanied by its own audio track played via PlaySong.
;           Sets cutScenePlayed = 1 so the cutscene is never shown again in
;           the same session.
;           Also calls PlayRunwayCutscene, showing the player's plane taking off.
;           On any file error, prints an error message and exits early.
;
; Entry   : cutScenePlayed - 0 (checked before)
;           FileLoadngPlay1/2/3Bmp - filenames of the three story screens.
;           InstFile1/2/3      - filenames of the three audio tracks.
;
; Exit    : cutScenePlayed = 1.
;           Three BMP+audio pairs shown and played in order.
;           secondBuffer updated and blitted for each frame via TransitionBuffer.
;           FileError = 1 on any file open failure (error message printed).
;           DX preserved.
;
; Arbitrary numbers explained:

```

```

; 320 - full screen width in pixels (VGA Mode 13h).
; 200 - full screen height in pixels (VGA Mode 13h).
; Both values fill the entire display so each cutscene image covers the
; whole screen with no border.
;
=====
=====
;
=====
=====
; PROC PlayRunwayCutscene
; -----
; Purpose : Display a runway scene as the final frame of the intro
;          cutscene, giving the impression the player's aircraft is taking off.
;          Holds the image on screen and shows the player taking off.
;
; Entry   : AircraftRunwayBmp - filename of the runway scene BMP asset.
;          Player's Aircraft spries in different sizes.
;          secondBuffer      - off-screen render target.
;
; Exit    : Runway BMP rendered to secondBuffer and blitted to the screen. Player
taking off.
;
;
=====
=====
;
=====
=====
; PROC DrawFromPixelArray
; -----
; Purpose : Iterate through a pixel-triplet array (X, Y, Color word triplets)
;          and plot each valid, in-bounds pixel into secondBuffer.
;          Skips any pixel whose X or Y falls outside the screen boundary.
;          Stops at three consecutive zero words: 0,0,0.
;
; Entry   : [bp+4] = offset of the pixel-triplet array to draw.
;          Each entry is 3 words: [X word][Y word][Color word].
;          Array must be terminated by 0, 0, 0.
;          secondBuffer - off-screen render target in its own segment.
;

```

```

; Exit : All in-bounds, non-sentinel pixels written to secondBuffer.
;       The 2-byte stack argument is cleaned up by the procedure (ret 2).
;       AX, BX, CX, DX, DI, SI, ES preserved.
;
;
=====
=====
=====
=====
; PROC HandleEnemy
; -----
; Purpose : Per-frame handler for Enemy 1. Future being for all.
;          Manages the full enemy lifecycle: left-side timer, fire-death,
;          normal flight, disappear trigger, shrink animation, and respawn.
;
; Entry : enemyLeft    - 1 if enemy spawned on the left side.
;        enemyLeftTimer - frames elapsed since left-side spawn.
;        EnemyDeadByFire - 1 if enemy was shot by the player.
;        isDead        - 1 if enemy has finished its death sequence.
;        DisappearNow   - 1 if shrink/disappear sequence should run.
;        horizonLine    - current horizon Y coordinate.
;
; Exit : Enemy drawn or removed from screen.
;       isDead, enemyLeft, enemyLeftTimer, DisappearNow updated as needed.
;       On respawn: all Enemy 1 state variables reset,
;       new position assigned via SpawnEnemyOnRandomCorner.
;
; Arbitrary numbers explained:
; 70 - A number fitting until the enemy should disappear, allows for speed to match
right.
; 147 - target X before triggering Disappear.
; 20 - target Y before triggering Disappear.
; 339 - target X before triggering Disappear (For Left).
;
=====
=====
-----
;
=====
=====
; PROC SyncAllEnemies

```

```

; -----
; Purpose : After Enemy 1's Normal sprite is moved, move the same
;           positional delta to the Close, Middle, and Far size sprite's array so
;           all four sprites stay at the same relative screen location.
;
; Entry   : EnemyNormal[0] - current X of the reference (normal-size) sprite.
;           EnemyNormal[2] - current Y of the reference sprite.
;           EnemyClose, EnemyMiddle, EnemyFar - last-synced positions of the
;           smaller variants (each is a pixel-triplet array).
;
; Exit    : EnemyClose, EnemyMiddle, EnemyFar X and Y fields shifted by the
;           same delta that was applied to EnemyNormal since last sync.
;           Registers Preserved.
;
;
=====
=====
----
;
=====
=====
; PROC resetVariabels
; -----
; Purpose : Restore Enemy 1 to its canonical starting state ready for a
;           fresh spawn. Copies the fixed coordinate table back
;           into the live EnemyNormal array, resets all animation counters, etc..
;
; Entry   : EnemyNormalFixed - read-only reference array with original coords.
;
; Exit    : EnemyNormal      - overwritten with EnemyNormalFixed fixed values.
;           important related values like:
;           disappearLevel   - 0 (no disappear frames consumed).
;           DisappearNow     - 0 (disappear sequence not active).
;           cxMODE           - 0 (size stage counter reset).
;           CxSizeTimer      - 1 (shift-register timer reset to initial state).
;           DI preserved.
;
;
=====
=====
----

```

```

;
=====
=====
; PROC setPosToFixedArray
; -----
; Purpose : Copy every (X, Y, Color) triplet from EnemyNormalFixed into
;           EnemyNormal, effectively restoring the sprite to its original
;           design-time position. Stops at the sentry '0, 0, 0'.
;
; Entry  : EnemyNormalFixed - source array of word triplets, terminated by
;           three consecutive zero words.
;           EnemyNormal      - destination array of the same layout.
;
; Exit   : EnemyNormal contains an exact copy of EnemyNormalFixed up to and
;           including the sentinel.
;           SI, DI, AX preserved.
;
; Arbitrary numbers explained:
; 6 - byte size of one pixel triplet (3 words x 2 bytes = 6 bytes);
; SI and DI are advanced by 6 after each copied entry.
;
=====
=====
----
;
=====
=====
; PROC EnemyDisappearStep
; -----
; Purpose : Each frame during Enemy 1's shrink sequence, select the correct
;           size-variant sprite by reading the time-based CX counter, and
;           load its offset into DI for the caller to draw.
;           When CX exceeds 400 the enemy is marked dead.
;
; Entry  : cxMODE / CxSizeTimer - consumed by SetCXbyTime to produce CX.
;
; Exit   : DI = offset of the sprite to draw this frame:
;           CX <= 100 = EnemyNormal (full size, still close)
;           CX <= 200 = EnemyClose (slightly smaller)
;           CX <= 300 = EnemyMiddle (medium distance)
;           CX <= 400 = EnemyFar (small / far)

```

```

;      CX > 400 = isDead set to 1, DI unchanged.
;      AX, BX, CX preserved.
;
; Arbitrary numbers explained:
; 100, 200, 300, 400 - CX thresholds corresponding to cxMODE values 1-4
;      (each mode = 100 units); four stages of shrink before
;      the sprite is considered gone.
;
=====
=====
----
;
=====
=====
; PROC MoveEnemyToBoundry
; -----
; Purpose : Move every pixel in the given enemy sprite array one step toward
;      a target (X, Y) screen position. Uses a shift-register timer
;      (EnemyStayAtPlaceTimer) to add a brief per-axis pause.
;
; Entry  : [bp+4] = target Y screen coordinate.
;      [bp+6] = target X screen coordinate.
;      [bp+8] = DI - offset of the pixel-triplet array to move.
;      [bp+10] = AX - initial step size (overridden by random if ax = 0).
;      EnemyStayAtPlaceTimer - shift-register that gates movement.
;
; Exit   : Every pixel's X Y in the array shifted one step toward target.
;      isLeft - set to 1 if moving left, 0 if moving right.
;      isForawrd - set to 1 when X and Y both within  $\pm 10$  of target.
;
;      All registers preserved.
;
;
=====
=====
----
;
=====
=====
; PROC WaitToDisappear
; -----

```



```

; Purpose : Test whether Enemy 1's current position is within  $\pm 4$  pixels of
;           the designated disappear point (X, Y passed on the stack).
;           If so, set DisappearNow and return carry set to signal the caller
;           that the disappear sequence should begin.
;
; Entry  : [bp+4] = target Y (horizon-relative disappear Y).
;           [bp+6] = target X (147 - the fixed screen disappear X).
;           EnemyNormal[0] - current X of the sprite.
;           EnemyNormal[2] - current Y of the sprite.
;
; Exit   : Carry set = sprite is within  $\pm 4$  pixels of target; DisappearNow = 1.
;           Carry clear = sprite has not yet reached the target.
;           AX, BX preserved.
;
; Arbitrary numbers explained:
; 4 - tolerance in pixels; the enemy is considered "at" the disappear
;     point when both  $|dx| \leq 4$  and  $|dy| \leq 4$ , avoiding exact-match
;     dependence when movement steps may overshoot by a pixel.
;
=====
=====
----
;
=====
=====
; PROC DrawAnything
; -----
; Purpose : Thin wrapper around OpenShowBmp that displays a BMP file using
;           the current values of BmpLeft, BmpTop, BmpWidth, BmpHeight, and
;           FileNamePtr. Prints an error message to the console if the file
;           cannot be opened.
;
; Entry  : FileNamePtr - offset of null-terminated BMP filename.
;           BmpLeft   - screen X of the top-left corner.
;           BmpTop    - screen Y of the top-left corner.
;           BmpWidth  - pixel width of the BMP.
;           BmpHeight - pixel height of the BMP.
;
; Exit   : BMP pixels written to secondBuffer via OpenShowBmp.
;           OR FileError (printed error) - set to 1 on open failure (set inside
OpenShowBmp).

```

```

;
; Modifies: FileError.
;
=====
=====
----
;
=====
=====
; PROC SpawnEnemyOnRandomCorner
; -----
; Purpose : Initialise Enemy's starting position for a new wave.
;     Always places the sprite at Y = 140 (near the horizon).
;     Randomly selects left-side (enemyLeft = 1, no X offset applied)
;     or right-side (X += 269) spawn using a 0/1 random coin-flip.
;
; Entry  : EnemyNormal - must have been reset via resetVariabels before call.
;
; Exit   : EnemyNormal Y field incremented by 140 for every pixel.
;     If right spawn: EnemyNormal X field incremented by 269 for every
;     pixel, placing the sprite near the right edge of the screen.
;     If left spawn: enemyLeft = 1; X is left at its reset position
;     (near the left edge).
;     AX, BX, DX, CX preserved.
;
;
;
=====
=====
----
;
=====
=====
; PROC AddXToOffsetInArray
; -----
; Purpose : Add a signed word value to one field of every (X, Y, Color)
;     triplet in a pixel array. The field is selected by a byte offset
;     (0 = X, 2 = Y, 4 = Color). Iteration stops at the sentinel
;     triplet (three consecutive zero words).
;
; Entry  : [bp+4] = signed word delta to add.
;     [bp+6] = byte offset within each triplet (0, 2, or 4).

```

```

;      [bp+8] = SI - offset of the pixel-triplet array.
;
; Exit   : Every non-sentinel entry's chosen field incremented by delta.
;      SI, AX, BX preserved.
;
;
=====
=====
----
;
=====
=====
; PROC SetCXbyTime
; -----
; Purpose : Advance Enemy's size-stage counter (cxMODE) using a
;      shift-register "timer" (CxSizeTimer), then compute
;      CX = cxMODE * 100. Caps cxMODE at 4 (after 4 [5]) (four shrink stages).
;
; Entry   : CxSizeTimer - word used as a shift register; carry from SHL
;      gates each cxMODE increment.
;      cxMODE    - current size stage (0-4).
;
; Exit    : CX = cxMODE * 100 (range 0-400).
;      cxMODE possibly incremented (max 4).
;      CxSizeTimer updated.
;
;
=====
=====
----
;
=====
=====
; PROC RandomByCsW
; -----
; Purpose : Generate a random integer in the range (BX, DX) using a
;      combination of the BIOS timer and XOR of opcodes in cs.
;
;
; Given a draw range in bx, dx
; 1. We subtract from the largest to the smallest to get a range from 0
; 2. We take a hundredth of a second from the clock

```

```

; 3. We take 2 opcodes from cs using di, mix one of them and do an xor between
them together with the address di
; 4. We advance in CS to the next position with di
; 5. We do an xor between the opcodes and the hundredth of a second
; 6. We discard bits that are not in the range (and with a mask of the range)
; 7. If still not in the range, we jump to 2 and do it again.
; 8. Otherwise, we are done, we add the low end of the range to the number drawn
and return in ax a number that is muffled.

```

```

;
; Entry : BX = inclusive lower bound.
;        DX = exclusive upper bound.
;        (DX must be > BX)
;
; Exit  : AX = random value in [BX, DX).
;        Position (CS variable) updated for next call.
;        BX, DX, SI, DI, ES preserved.
;
;

```

```

=====
=====

```

```

----
;
=====
=====

```

```

; PROC make_mask_right
; -----
; Purpose : Use a bitmask to make the generated random num in range (bits out of
range)
;
; Entry : DX = range width (DX_original - BX from RandomByCsW).
;
; Exit  : SI = mask
;
;

```

```

=====
=====

```

```

----
;
=====
=====

```

```

; PROC HandleAltitude

```

```

; -----
; Purpose : Per-frame altitude manager. First checks whether the player has
;           reached a dangerous altitude (triggering warnings or game-over via
;           checkAltitude), then draws the altitude indicator HUD element
;           (a small BMP strip) in the bottom-left corner of the screen.
;
; Entry   : horizonLine - current horizon Y position; read by checkAltitude
;           to determine if the player is too high or too low.
;           AltitudeBmp - null-terminated filename of the altitude HUD BMP.
;           secondBuffer - off-screen render target written by OpenShowBmp.
;
; Exit    : Altitude danger checks performed (warnings/game-over if needed).
;           Altitude HUD BMP rendered to secondBuffer.
;           FileError    - set to 1 by OpenShowBmp if the BMP cannot be opened.
;
;
=====
=====
;
=====
=====
; PROC checkAltitude
; -----
; Purpose : Each frame, check whether the horizon line has moved so far up
;           or down that the player is at a dangerous altitude, and trigger
;           an audio warning accordingly.
;
; Entry   : horizonLine - current Y position of the horizon in screen pixels.
;
; Exit    : If horizonLine <= 10 : Crash.
;           If horizonLine >= 190 : Altitude Ripped Aircraft Airframe.
;
;
=====
=====
-----
;
=====
=====
; PROC SpeakerOn
; -----

```

```

; Purpose : Program PIT with the given frequency divisor and gate the PC Speaker
output on.
;
; Entry  : AX = desired frequency in Hz (must be > 0).
;
; Exit   : PC Speaker emitting the tone at AX Hz.
;         All registers preserved.
;
;
=====
=====
----
;
=====
=====
; PROC SpeakerOff
; -----
; Purpose : Silence the PC Speaker by clearing speaker-enable bits in port 61h.
;
; Entry   : None.
;
; Exit    : PC Speaker muted.
;         AX preserved.
;
;
=====
=====
----
;
=====
=====
; PROC flushKeys
; -----
; Purpose : Drain the BIOS keyboard buffer so that any keys pressed during
;         a previous screen (menu, cutscene, explosion) do not leak into
;         the next input-polling cycle.
;
; Entry   : None.
;
; Exit    : BIOS keyboard buffer empty.
;         All registers preserved.

```

```

;
;
=====
=====
----
;
=====
=====
; PROC HandleSpeed
; -----
; Purpose : Per-frame speed manager. First checks whether the player has
;          reached a dangerous speed (see CheckSpeed), then draws the speed indicator
;          HUD element
;          (a small BMP strip) in the bottom-left corner of the screen.
;
; Entry  : CheckSpeed Entries (see CheckSpeed).
;          SpeedBmp   - null-terminated filename of the speed HUD BMP.
;          secondBuffer - off-screen render target written by OpenShowBmp.
;
; Exit   : Speed danger checks performed.
;          Speed HUD BMP rendered to secondBuffer.
;          FileError   - set to 1 by OpenShowBmp if the BMP cannot be opened.
;
;
=====
=====
;
=====
; PROC CheckSpeed
; -----
; Purpose : Each frame, read the speed-up and slow-down keys, update
;          planeSpeed, and call Stall if speed has dropped to minimum.
;
; Entry  : keys[]   - current keyboard state (1 = held, 0 = released).
;          planeSpeed - current forward speed (word, 1..40).
;
; Exit   : planeSpeed possibly incremented (speed-up key) or decremented
;          (either slow-down key).
;          Stall called if speed = 1 (handled inside Stall).
;          AX preserved.

```

```

;
; Arbitrary numbers explained:
; 39h - Scan code for SPACE BAR (speed up).
; 2Ah - Scan code for LEFT SHIFT (slow down).
; 36h - Scan code for RIGHT SHIFT (slow down).
; 40 - Maximum allowed plane speed- acts as an upper speed cap.
; 1 - Minimum plane speed- dropping below triggers a stall.
;
=====
=====
----
;
=====
=====
; PROC checkMainMenu
; -----
; Purpose : Poll the keyboard hardware port directly for a single keypress
;          on the main menu screen. Returns carry set for "Play", or sets
;          the Direction Flag (DF) to signal "Exit" to the caller.
;
; Entry  : None (reads port 60h directly).
;
; Exit   : Carry set = scan code 2 pressed (key '1' = Play).
;          DF set    = scan code 4 pressed (key '3' = Exit).
;          Carry clear, DF clear = any other key or key release.
;          AX, BX preserved.
;
; Arbitrary numbers explained:
; 60h - PC keyboard data port; holds the last scan code.
; 2 - Scan code for the '1' key (Play option).
; 4 - Scan code for the '3' key (Exit option).
;
=====
=====
----
;
=====
=====
; PROC DebugPaletteColors
; -----
; Purpose : Development utility. Draw a row of 16 color swatches (20x20

```



```

;    pixels each) starting at Y=5 in secondBuffer so that the first
;    16 VGA palette indices can be visually inspected.
;    Not called in the final game loop.
;
; Entry  : secondBuffer - off-screen frame buffer (must be initialised).
;
; Exit   : secondBuffer contain 16 filled rectangles, one per
;          palette index 0-15.
;          All registers preserved.
;
; Arbitrary numbers explained:
; 16 - Number of palette swatches to draw (indices 0-15).
; 20 - Width and height of each color swatch in pixels.
; 5  - Starting Y coordinate of the swatch row (leaves a small margin).
; 320 - Screen width in bytes per row (VGA Mode 13h).
;
=====
=====
----
;
=====
=====
; PROC _200MiliSecDelay
; -----
; Purpose : Busy-wait for approximately 200 milliseconds.
;
; Entry  : None.
;
; Exit   : Approximately 200 ms have elapsed.
;          All registers preserved.
;
; Arbitrary numbers explained:
; 1000 - Outer loop count.
; 600  - Inner loop count. 1000 * 600 = 600,000 iterations.
;        At roughly 3,000,000 iterations per second on a 286-era machine
;        this produces ~200 ms of delay.
;
=====
=====
----

```

```

;
=====
=====
; PROC _400MiliSecDelay
; -----
; Purpose : Busy-wait for approximately 400 milliseconds by calling
;           _200MiliSecDelay twice in sequence.
;
; Entry   : None.
;
; Exit    : Approximately 400 ms have elapsed.
;           No registers.
;
=====
=====
----
;
=====
=====
; PROC DrawMainMenu
; -----
; Purpose : Render the "Press Any Key" splash screen (L_2.bmp) to the
;           off-screen buffer covering the full 320x200 display.
;           Called once per frame during the pre-menu attract loop.
;
; Entry   : FileMainMenuShow - offset of the null-terminated filename "L_2.bmp".
;           secondBuffer    - destination off-screen frame buffer.
;
; Exit    : secondBuffer filled with the start screen image.
;           OR ileError = 1 and error message printed if file open fails.
;           All registers preserved.
;
; Arbitrary numbers explained:
; 320 - Full screen width in pixels (VGA Mode 13h).
; 200 - Full screen height in pixels (VGA Mode 13h).
; 0, 0 - Top-left corner; image covers the entire screen.
;
=====
=====
----

```

```

;
=====
=====
; PROC MainMenu
; -----
; Purpose : Display the main menu selection screen (LOADING.bmp - "1-PLAY,
;          3-EXIT") to the off-screen buffer. The caller is responsible
;          for blitting and waiting for a key press.
;
; Entry   : FileMainReady - offset of null-terminated filename "LOADING.bmp".
;          secondBuffer - destination off-screen frame buffer.
;
; Exit    : secondBuffer filled with the menu image.
;          FileError = 1 and error message printed if file open fails.
;          BX, CX, DX preserved.
;
; Arbitrary numbers explained:
; 320 - Full screen width in pixels.
; 200 - Full screen height in pixels.
; 0, 0 - Top-left origin; image covers the entire display.
;
=====
=====
----
;
=====
=====
; PROC reset_dsp
; -----
; Purpose : Reset the Sound Blaster DSP to a known idle state before issuing playback
commands.
;
; Entry   : DSP_RESET - SB DSP reset port.
;          DSP_RDSTAT - SB DSP read-data-available status port.
;          DSP_READ - SB DSP read data port.
;
; Exit    : DSP in reset/idle state.
;          All registers preserved.
;

```

```

;
=====
=====
----
;
=====
=====
; PROC PlaySong
; -----
; Purpose : Open a raw 8-bit PCM audio file and stream it to the Sound
;          Blaster DSP via direct DAC output commands (DSP command 10h).
;          Stops and returns with carry set if any key is pressed.
;          Stops and returns with carry clear when the file is exhausted.
;
; Entry  : [bp+4] = offset of null-terminated filename for the RAW audio file.
;          DSP_WRITE - SB DSP write port (EQU 22Ch).
;          DSP_RESET, DSP_READ, DSP_RDSTAT - SB base ports.
;          CHUNK_SIZE (EQU 400h = 1024) - bytes read per disk I/O call.
;          SAMPLE_DELAY (EQU 25) - inter-sample busy-wait iterations
;          to approximate the playback sample rate.
;
; Exit   : Carry set = user pressed a key during playback (early stop).
;          Carry clear = file played to completion normally.
;          file_handle closed on both exit paths.
;          All registers preserved.
;
;          Note: Depending on development the proc either can / has a variant
;          of calling other procs from its "break" from playing the song.
;
=====
=====
----
;
=====
=====
; PROC file_open
; -----
; Purpose : Open a file by name for read-only access using DOS INT 21h and
;          store the returned handle in FileHandle.
;          Sets FileError = 1 and FileFound = 0 on failure.
;          Sets FileFound = 1 and FileHandle = handle on success.

```

```

;
; Entry : FileNamePtr - word containing the offset of the null-terminated
;         filename string to open.
;
; Exit  : FileHandle = valid DOS file handle (on success).
;         FileFound  = 1 (success) or 0 (failure).
;         FileError  = 0 (success) or 1 (failure).
;         DX preserved.
;
; Arbitrary numbers explained:
; 3Dh - DOS function: open existing file.
; AL=0 - Open mode: read-only (no write access).
;
=====
=====
----
;
=====
=====
; PROC DrawGrassLines
; -----
; Purpose : Project three pairs of 3D waves lines endpoints through
;         the perspective transformation and draw each visible line to the
;         off-screen buffer using Bresenham. Lines below the horizon are
;         clipped (skipped). Gives the illusion of a receding landscape.
;
; Entry : tree2_x0/x1, tree3_x0/x1, tree4_x0/x1 - world-space X endpoints
;         for each line (word variables).
;         tree2_z / tree3_z / tree4_z - world-space Z depths.
;         horizonLine - current horizon Y in screen pixels.
;         color      - set to 6 internally (dark green/olive).
;
; Exit  : Up to 3 projected lines drawn to secondBuffer.
;         AX, BX, CX preserved.
;
; Arbitrary numbers explained:
; 6 - Blue color
;
;
=====
=====

```

```

----
;
=====
=====
; PROC OverflowFix
; -----
; Purpose : Make sure none of the coords exceed the legal values for drawing the
waves.
;
; Entry  : DI = projected screen X (may be outside 0-319).
;         SI = projected screen Y (may be above horizon).
;         horizonLine - minimum allowed Y value for ground geometry.
;
; Exit   : DI clamped to [0, 319].
;         SI clamped to [horizonLine, 199] (horizon enforced as minimum).
;         All other registers preserved.
;
; Arbitrary numbers explained:
; 319 - Maximum valid screen X (320 pixels wide).
; 0   - Minimum valid screen X.
;
=====
=====
----
;
=====
=====
; PROC DrawPlane
; -----
; Purpose : Each frame, select the correct player aircraft BMP based on the
;           currently held directional key and draw it to the off-screen
;           buffer.
;           Does nothing if GameOver is set.
;
; Entry  : keys[] - keyboard state array.
;         PlaneState - cached plane orientation (0-4).
;         GameOver - if 1, proc returns immediately without drawing.
;         All aircraft BMP filenames and dimensions defined in DATASEG.
;
; Exit   : secondBuffer updated with the correct plane sprite.
;         PlaneState updated to reflect currently held key.

```

```

;      DX preserved.
;
; Arbitrary numbers explained:
; 20h - Scan code for the 'D' key (bank right).
; 1Eh - Scan code for the 'A' key (bank left).
; 1Fh - Scan code for the 'S' key (pitch up / climb).
; 11h - Scan code for the 'W' key (pitch down / dive).
; 115 - Fixed BmpLeft: centres the fighter sprite horizontally within
;       the 320-pixel screen ( $320/2 - 42 \approx 115$  for an 83-px wide sprite).
; 88 - Fixed BmpTop: positions the sprite in the lower-centre of the
;       screen to simulate a first-person cockpit view.
; 83/85/84 - pixel widths of the respective directional BMP assets.
; 48/38/46/63/45 - pixel heights of the respective directional BMP assets.
;
=====
=====
----
;
=====
=====
; PROC Crash
; -----
; Purpose : Handle an unrecoverable crash event (terrain contact / extreme
;          altitude). Sets the GameOver flag, hides the plane sprite, and
;          delegates to GameOverProc for the explosion and end screen.
;
; Entry  : None.
;
; Exit   : GameOver = 1.
;         StopDrawingPlane called to remove the plane from rendering.
;         GameOverProc called to show explosion + Game Over screen.
;         Does not return to normal game flow.
;
=====
=====
----
;
=====
=====
; PROC GameOverProc
; -----

```

```

; Purpose : Execute the full game-over sequence:
;     1. Silence the PC Speaker.
;     2. Set GameOver flag.
;     3. Show and play the explosion at screen centre.
;     4. Display the G_O.bmp (Game Over) full-screen image.
;     5. Wait until the player presses any key to continue.
;
; Entry  : None.
;
; Exit   : GameOver = 1.
;         secondBuffer filled with G_O.bmp.
;         anyKeyPressed = 1 when proc returns (player confirmed) (from proc called -
unless closed dos or exited game by force).
;         DX preserved.
;
;
=====
=====
----
;
=====
=====
; PROC HearAndSeeExplosion1
; -----
; Purpose : Display the explosion BMP at a caller-specified position and
;         play the explosion sound effect (EXPL.raw) through the Sound
;         Blaster. Provides both visual and audio feedback for any
;         destruction event.
;
; Entry  : [bp+4] = BmpLeft (screen X of explosion sprite).
;         [bp+6] = BmpTop (screen Y of explosion sprite).
;         FileExplosionBmp - filename offset for "Explo.bmp".
;         FileExplosionSfx - filename offset for "EXPL.raw".
;
; Exit   : Explosion sprite written to secondBuffer and blitted to screen.
;         EXPL.raw played through Sound Blaster.
;         ~400 ms additional delay after sound.
;         AX, DX preserved.
;
; Arbitrary numbers explained:
; 160 - Fixed explosion sprite width in pixels.

```



```

; 80 - Fixed explosion sprite height in pixels.
;
=====
=====
----
;
=====
=====
; PROC TerrainWarning ; TO FIX - BAD! BLOCKS GAME!
; -----
; Purpose : Emit a repeating low-high two-tone beep via the PC Speaker to
;         warn the player that they are approaching the terrain altitude
;         limit.
;
; Entry  : None (uses PC Speaker directly via SpeakerOn/SpeakerOff).
;
; Exit   : PC Speaker left silent after the final beep cycle.
;         AX, CX, DX preserved.
;
; Arbitrary numbers explained:
; 3      - Number of beep pairs to play; enough for an urgent warning
;         without blocking the game loop too long.
; 280    - Low tone frequency in Hz: a deep, attention-grabbing tone.
; 880    - High tone frequency in Hz: a sharp, urgent contrast tone.
; 800    - Outer loop count for the low-tone duration section.
; 800    - Inner loop count:  $800 \times 800 = 640,000$  cycles  $\approx \sim 100$  ms at 6 MHz.
; 400    - Outer loop count for the high-tone section (shorter = higher urgency).
; 100    - Silence outer loop count between low and high tones.
; 400    - Inner loop for silence period.
; 300    - Outer loop count for the pause between beep pairs.
; 800    - Inner loop for the between-pair pause.
;
=====
=====
----
;
=====
=====
; PROC Stall ; TO FIX! - BAD!
; -----
; Purpose : Simulate an aerodynamic stall when planeSpeed drops to 1.

```

```

; Forces the aircraft into a nose-down attitude, lowers the horizon
; (reduces altitude), plays the stall sound effect, and triggers
; GameOver if the horizon hits the minimum safe floor.
;
; Entry : planeSpeed - current speed (1 = minimum / stall threshold).
; horizonLine - current horizon Y in screen pixels.
; altitude - current altitude value.
; FileStallSfx - filename of "stl.raw" stall audio clip.
;
; Exit : If planeSpeed > 1: returns immediately (no stall).
; If planeSpeed = 1:
; horizonLine -= 8 (aircraft loses altitude rapidly).
; horizonLine floored at 10 (minimum safe altitude).
; altitude -= 7 if floor was reached.
; PlaneState = 4 (nose-down visual state).
; Stall sound played.
; GameOver set and GameOverProc called if horizonLine <= 10.
; All registers preserved.
;
; Arbitrary numbers explained:
; 1 - Minimum planeSpeed before stall activates.
; 8 - Horizon drop per stall frame: rapid altitude loss to feel urgent.
; 10 - Minimum horizonLine (terrain floor); below this = fatal crash.
; 7 - Altitude decrement when floor is hit (matches the rapid drop feel).
; 4 - PlaneState value for nose-down attitude (triggers F_D.bmp).
;
=====
=====
----
;
=====
=====
; PROC MoveUp
; -----
; Purpose : Move the player aircraft one unit upward (increase altitude).
; Adjusts the horizon line, increments altitude, and shifts both
; the enemy sprite and the in-flight missile array by +1 Y so they
; appear to move with the world.
;
; Entry : altitude - current altitude
; horizonLine - screen Y of the horizon.

```

```

;      EnemyNormal - enemy pixel-triplet array.
;      Missile_XXXS - missile pixel-triplet array.
;
; Exit  : altitude  += 1.
;      horizonLine += 1.
;      All Y fields in EnemyNormal += 1.
;      All Y fields in Missile_XXXS += 1.
;      AX preserved.
;
;
=====
=====
----
;
=====
=====
; PROC CheckAndMoveNew
; -----
; Purpose : Each frame, pull the keys[] array for the four directional keys
;      and call the corresponding movement procedure for each held key.
;      Multiple keys can be held simultaneously.
;
; Entry  : keys[] - interrupt-updated keyboard state array (1=held, 0=released).
;
; Exit   : MoveDown, MoveUp, MoveLeftNew, or MoveRight called as appropriate.
;      AX, BX preserved.
;
; Arbitrary numbers explained:
; 11h - Scan code for 'W' key (pitch down / dive).
; 1Fh - Scan code for 'S' key (pitch up / climb).
; 1Eh - Scan code for 'A' key (turn/slide left).
; 20h - Scan code for 'D' key (turn/slide right).
;
=====
=====
----
;
=====
=====
; PROC MoveDown
; -----

```

```

; Purpose : Move the player aircraft one unit downward (decrease altitude).
;     Decrements horizon and altitude, then shifts both the enemy sprite
;     and missile array by -1 Y so they go by the world.
;
; Entry  : altitude  - current altitude (must be > 0 to descend safely).
;     horizonLine - current horizon screen Y.
;     EnemyNormal - enemy pixel-triplet array.
;     Missile_XXS - missile pixel-triplet array.
;
; Exit   : altitude  -= 1.
;     horizonLine -= 1.
;     All Y fields in EnemyNormal -= 1.
;     All Y fields in Missile_XXS -= 1.
;     If altitude was 0: Crash is called.
;
;
=====
=====
----
;
=====
=====
; PROC MoveForwards
; -----
; Purpose : Each frame, advance all four landscape Z-depth values by a
;     speed-dependent step so that ground objects appear to rush toward
;     the player.  Objects that pass too close (Z < 245) are
;     teleported to a far distance via KillObject to create an
;     infinite-looping landscape.
;
; Entry  : planeSpeed - current forward speed (1-40).
;     tree1_z / tree2_z / tree3_z / tree4_z - world Z depths of the
;     four ground-line anchor points.
;
; Exit   : Each tree Z decremented by (planeSpeed * 2).
;     Trees whose Z crossed 230 are reset to a far position via
;     KillObject.
;     AX preserved.
;

```

```

;
=====
=====
----
;
=====
=====
; PROC MoveRight
; -----
; Purpose : Each frame the 'D' key is held, slide all landscape ground lines
;         left by 3 pixels (simulating rightward camera pan) and shift both
;         the enemy sprite and the missile array left by 4 pixels.
;         After 600 consecutive right-movement frames, performs a "FullSpin"
;         reset that teleports all tree X coordinates to new far-right
;         starting positions, creating an infinite-loop landscape.
;
; Entry   : MoveRightCnt - accumulated right-movement frame counter.
;         tree2/3/4 _x0, _x1 - world X endpoints of the three ground lines.
;         EnemyNormal - enemy pixel-triplet array.
;         Missile_XXXS - missile pixel-triplet array.
;
; Exit    : tree2/3/4 _x0/_x1 each decremented by 3 (normal case).
;         All X fields in EnemyNormal decremented by 4.
;         All X fields in Missile_XXXS decremented by 4.
;         MoveRightCnt incremented (normal case) or reset to 0 (spin case).
;         On FullSpin: tree X endpoints reset to hardcoded far positions.

;         AX preserved.
;
;
=====
=====
----
;
=====
=====
; PROC MoveLeftNew
; -----
; Purpose : Each frame the 'A' key is held, slide all landscape ground lines
;         right by 3 pixels (simulating leftward camera pan) and shift the

```

```

;      enemy sprite right by 4 pixels.After 600 consecutive right-movement frames,
performs a "FullSpin"
;      reset that teleports all tree X coordinates to new far-left
;      starting positions, creating an infinite-loop landscape.
;
; Entry  : tree2/3/4 _x0, _x1 - world X endpoints of the three ground lines.
;      EnemyNormal - enemy pixel-triplet array.
;      Missile_XXXS - missile pixel-triplet array.
;              moveLeftCnt - for infinte spin.
;
; Exit   : tree2/3/4 _x0/_x1 each incremented by 3.
;      All X fields in EnemyNormal  incremented by 4.
;      All X fields in Missile_XXXS decremented by 4
;      All registers preserved.
;
;
=====
=====
----
;
=====
=====
; PROC KillObject
; -----
; Purpose : Teleport a landscape object to a far-away position so it can
;      re-enter the scene from the distance, creating the illusion of
;      an infinite looping landscape. Sets both a new Z depth and a
;      new Y (horizon-relative) value via the object's data structure.
;
; Entry  : [bp+4] = SI - pointer to the object's data block.
;      Word at [SI+2] = Y coordinate to reset.
;      Word at [SI+4] = Z depth to reset.
;
; Exit   : [SI+4] = 1400 (new far Z depth).
;      [SI+2] = 100 (new Y position near the horizon).
;      SI preserved.
;
;
=====
=====
----

```

```

;
=====
=====
; PROC fillAround
; -----
; Purpose : Each frame, clear the off-screen buffer by painting the sky
;           region (above the horizon) and the ground region (below the
;           horizon) with solid colors. This serves as the background
;           clear before all other objects are drawn on top.
;
; Entry   : horizonLine - current Y coordinate of the horizon in screen pixels.
;           secondBuffer - off-screen frame buffer to fill.
;
; Exit    : secondBuffer top filled with color 9 (sky).
;           secondBuffer bottom filled with color 4 (ground).
;           AX, BX, DX preserved.
;
; Arbitrary numbers explained:
; 320     - Screen width in bytes per row (VGA Mode 13h).
; 9       - VGA palette index for the sky color.
; 4       - VGA palette index for the ground color.
; 64000   - Total bytes in a 320x200 VGA frame (320 * 200); used to derive
;           the ground region size = 64000 - (horizonLine * 320).
;
=====
=====
----
;
=====
=====
; PROC Calc3D
; -----
; Purpose : Transform a 3D world coordinate to a presentable (on a 2d screen) point,
;           with only X,Y, without a third coordinate, while obviously maintaining the third
;           demension.
;           The formula applied is:
;           screenX = (worldX * FOCAL / Z) + 160
;           screenY = (worldY * FOCAL / Z) + 100
;
; Entry   : AX = world X coordinate (signed word).
;           BX = world Y coordinate (signed word).

```

```

;      CX = world Z depth (if as big as z=4 [further = smaller Z], its "behind", which is
coords 0, 0).
;      CONST_FOCAL = focal length constant (200).
;      Mid Sceen Values: 160 (320/2) and 100 (200/2) so result doesn't appear at
;      corner.
;
; Exit   : DI = projected screen X.
;      SI = projected screen Y.
;      If CX <= 4 (point is too close to camera):
;      DI = 0, SI = 0.
;      AX BX CX preserved.
;
;
=====
=====
----
;
=====
=====
; PROC DrawPoints
; -----
; Purpose : Paint all pixel addresses accumulated by Bresenham_GetPoints into
;      the off-screen buffer (secondBuffer) using the current color value.
;
; Entry   : points[] - array of word-sized flat buffer offsets for each
;      pixel to draw (populated by Bresenham_GetPoints).
;      pointsCount - number of valid entries in points[].
;      color      - byte color index to paint at each address.
;      secondBuffer - destination off-screen frame buffer.
;
; Exit    : Drawn pixels at secondBuffer.
;
; Arbitrary numbers explained:
; 2 - Each entry in points[] is a word (2 bytes).
;
=====
=====
----
;
=====
=====

```



```

; PROC FillWorld
; -----
; Purpose : Fill a contiguous region of the off-screen buffer with a single
;          solid color using REP STOSB. Used by fillAround to paint the
;          sky and ground regions each frame.
;
; Entry   : [bp+8] = start byte offset within secondBuffer.
;          [bp+6] = fill color (byte, VGA palette index).
;          [bp+4] = number of bytes to fill (pixel count).
;
; Exit    : secondBuffer filled (cleared).
;          AX, ES, DI, BP preserved.
;
;
=====
=====
----
;
=====
=====
; PROC Bresenham_GetPoints
; -----
; Purpose : Implements the Bresenham line algorithm to calculate all pixel
;          addresses along a line between (x0, y0) and (x1, y1). The offsets
;          are calculated for a 320x200 screen and stored in an array.
;
; Entry   : x0, y0 - Starting coordinates.
;          x1, y1 - Ending coordinates.
;          random noise?
;
; Exit    : points      - Array filled with memory offsets (y*320 + x).
;          pointsCount - Number of points calculated and stored.
;          AX, BX, CX, DX, SI, DI preserved.
;
; Note: Can implement random noise to the line on the y axis. Only up to y-1.
;
=====
=====
;
=====
=====

```

```

; PROC TransitionBuffer
; -----
; Purpose : Copy the completed off-screen frame (secondBuffer) to the VGA
;          display memory (segment 0A000h) in a single burst transfer,
;          implementing double-buffered rendering to eliminate flicker.
;
; Entry   : secondBuffer - fully rendered off-screen frame (64,000 bytes).
;
; Exit    : VGA display memory updated with the new frame.
;          DS, ES, SI, DI, CX preserved.
;
; Arbitrary numbers explained:
; 0A000h - VGA Mode 13h.
; 32000 - Word count for REP MOVSW: 32,000 words * 2 bytes = 64,000 bytes,
;         which is the full 320x200 frame.
;
=====
=====
----
;
=====
=====
; PROC OpenShowBmp
; -----
; Purpose : High-level BMP loader pipeline. Opens the file, reads the
;          54-byte header, reads and installs the 256-color palette, then
;          draws the pixel data to secondBuffer at the position described
;          by BmpLeft/BmpTop/BmpWidth/BmpHeight.
;          Aborts the pipeline if opening file fails.
;
; Entry   : FileNamePtr - offset of the null-terminated BMP filename.
;          BmpLeft, BmpTop, BmpWidth, BmpHeight - destination rect.
;
; Exit    : secondBuffer updated with the BMP image pixels.
;          FileError = 1 if OpenBmpFile failed (file not found etc.).
;          File handle closed before return on all paths.
;
=====
=====
----

```

```

;
=====
=====
; PROC OpenBmpFile
; -----
; Purpose : Open the BMP file named by FileNamePtr for read-only access
;          using DOS INT 21h and store the resulting handle in FileHandle.
;          Sets FileError = 1 on any DOS error (file not found, access
;          denied, etc.).
;
; Entry  : FileNamePtr - word containing the offset of the null-terminated
;          filename to open.
;          input dx filename to open (in proc).

;
; Exit   : FileHandle = valid DOS file handle (on success).
;          FileError = 0 (success) or 1 (failure).
;          DX preserved.
;
; Arbitrary numbers explained:
; 3Dh - DOS function number: open existing file.
; AL=0 - Open access mode: 0 = read-only.
;
=====
=====
----
;
=====
=====
; PROC CloseBmpFile
; -----
; Purpose : Close the currently open BMP file using DOS INT 21h to release
;          the file handle stored in FileHandle.
;
; Entry  : FileHandle - valid DOS file handle previously returned by
;          OpenBmpFile.
;
; Exit   : File handle released to DOS.
;          AH BX modified internally restored - low-level routine, caller ok with it).
;
; Arbitrary numbers explained:

```

```

; 3Eh - DOS function number: close file handle.
;
=====
=====
----
;
=====
=====
; PROC ReadBmpHeader
; -----
; Purpose : Read the 54-byte standard BMP file header from the open file
;           into the Header buffer. This advances the DOS file pointer past
;           the header so the next read will begin at the palette data.
;
; Entry   : FileHandle.
;           Header   - 54-byte buffer in DATASEG.
;
; Exit    : Header[0..53] filled with BMP header bytes - Read 54 bytes the Header.
;           DOS file pointer advanced by 54 bytes.
;           CX, DX preserved.
;
; Arbitrary numbers explained:
; 3Fh - DOS function number: read from file handle.
; 54 - Standard BMP file header size in bytes (BITMAPFILEHEADER =
;      14 bytes + BITMAPINFOHEADER = 40 bytes = 54 total).
;
=====
=====
----
;
=====
=====
; PROC ReadBmpPalette
; -----
; Purpose : Read the BMP palette from the open file into the Palette buffer.
;           Must be called after ReadBmpHeader so the file pointer is positioned
;           after the header.
;
; Entry   : FileHandle - valid open DOS file handle.
;           Palette    - 1024-byte (400h) buffer in DATASEG.
;

```

```

; Exit   : Palette[0..1023] filled with palette color indexes.
;        DOS file pointer advanced by 1024 bytes.
;        CX, DX preserved.
;
; Arbitrary numbers explained:
; 3Fh - DOS function number: read from file handle.
; 400h - 1024 bytes: 256 palette entries * 4 bytes each.
;
;
=====
=====
----
;
=====
=====
; PROC CopyBmpPalette
; -----
; Purpose : Program the VGA with the 256-color palette loaded from the BMP file.
;
; Entry   : Palette[0..1023] 1024B by ReadBmpPalette
;
; Exit    : All 256 VGA entries updated with the BMP palette colors.
;          CX, DX preserved.
;
; Notes   : Will move out to screen memory the Colors
;           video ports are 3C8h for number of first Color_1
;           and 3C9h for all rest
;
;
=====
=====
----
;
=====
=====
; PROC ShowBMP
; -----
; Purpose : Read BMP pixel rows from the open file and store in the
;           correct position in secondBuffer.
;
; Entry   : BmpLeft, BmpTop      - top-left destination in secondBuffer.

```

```

; BmpWidth, BmpHeight    - dimensions of the image.
; secondBuffer           - destination off-screen frame buffer.
;                         ScrLine                - one line to show on
screen saved.
;
; Exit  : secondBuffer updated with BMP pixel data.
;       CX preserved.
;
;
=====
=====
----
;
=====
=====
; PROC ShowAxDecimal
; -----
; Purpose : Write on screen the value of ax (decimal)
;          the practice :
;                      Divide AX by 10 and put the Mod on stack
;          Repeat Until AX smaller than 10 then print AX (MSB)
;          then pop from the stack all what we kept there and show it.
;
; Entry  : AX
;
; Exit   : Screen
;
;          All registers restored.
;
;
=====
=====

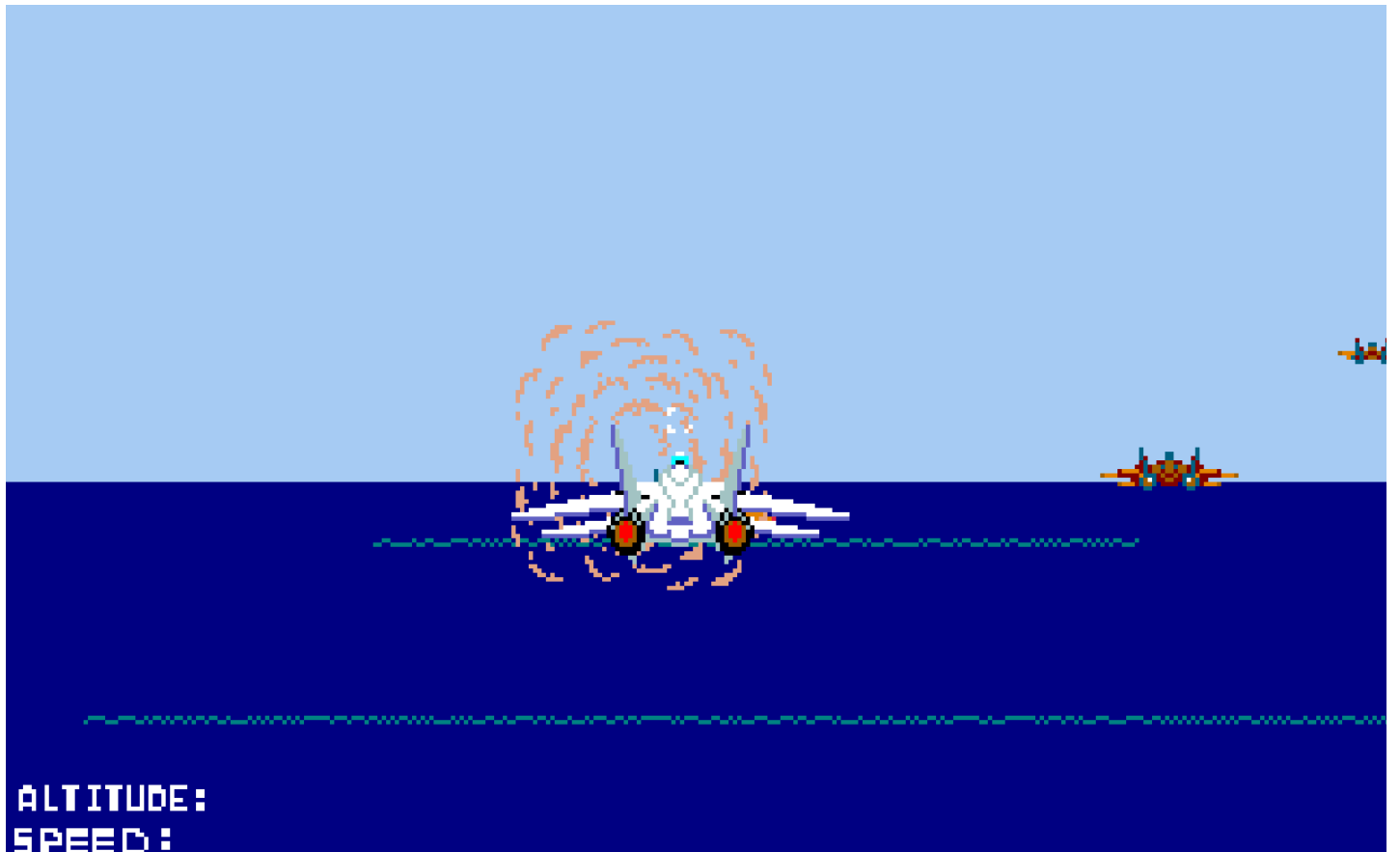
```

דוגמאות הרצה (צילומי מסך)











סיכום אישי

בפרויקט שלי, משחק תלת ממד בהשארת משחק הארקיד "After Burner", הצלחתי להגיע להישגים מרשימים בכתיבת קוד בשפת אסמבלי 8086, 16 ביט.

העבודה על הפרויקט הייתה עבורי מסע לתוך קרבי המחשב עצמו, שכן לא רק למדתי כיצד בונים משחק אמיתי מאפס, אלא עשיתי זאת באסמבלי, בשפה מאתגרת מאוד שמדברת ישירות עם חומרת המחשב.

במהלך הפיתוח נחשפתי לעולם הגרפיקה הממוחשבת ושיטות הרינדור. נהניתי במיוחד ליישם אשליות אופטיות המדמות תנועה במרחב תלת-ממדי על גבי מסך דו-ממדי, כגון שדה ראייה (Field of View) יצירת עולם דינמי הנע סביב שחקן סטטי. כדי להבטיח תצוגה חלקה ונקייה מריצודים, הטמעת את שיטת ה Double Buffer הכוללת כתיבה למקטע זיכרון זמני, Buffer, והעתקתו המהירה למקום המתאים בזיכרון ES בסיום כל פריים.

בנוסף, הקדמתי את החומר הנלמד וחקרתי באופן עצמאי את אלגוריתם Bresenham לציור קווים. כתיבת האלגוריתם בעצמי, עוד לפני שקיבלנו אותו כקוד מוכן, חיזקה אצלי את השליטה במתמטיקה של הגרפיקה. חקרתי גם טכניקות מתקדמות כמו Raycasting ואף שלא נכנסו לפרויקט הנוכחי, הידע שצברתי עליהן והתוכנה שערכתי עם שיטה זו פתחה לי אופקים חדשים.

יתר על כן, עניין אותי מאוד עניין האויב, התזוזה שלו, התקיפה, ויירוט האויב על ידי השחקן עצמו. חיכיתי מאוד לחלק הזה בפרויקט, כי כשפיתחתי אויב, פיתחתי משחק עובד.

דבר אחד שבניגוד לציפיות שלי עניין אותי מאוד הוא עניין המוזיקה במשחק. מוזיקת הרקע ואפקטי היריות של מטוס השחקן מרשימים ומעניינים בעבודה עם תדר אוויר בלבד, אך יתרה על כך, הצלחתי לבנות תוכנה שיכולה לנגן שיר באיכות גבוהה לתקופת המעבד 8086 (הכוללת אקורדים, ומספר צלילים ב"אותו" זמן).

כמו כן, חיבור פעולות כמו ניגון מוזיקת הרקע והמקלדת לטיימר האסינכרוני להפיכת הפעולות לאסינכרוניות עניין אותי מאוד, שכן אפשר להשתמש בדבר אחד (טיימר) עבור דברים שונים לחלוטין (מקלדת ומוזיקה).

בנוסף לנראה לעיל בעניין הקוד, גם כתיבת תיק פרויקט היווה תהליך משמעותי עבורי. למדתי כיצד לנהל משחק, מראש ובדיעבד, לעמוד בלוחות זמנים שהוצבו מראש, ועל סיכום כל הקשור בפרויקט, מסביבת העבודה, למשתנה הכי פחות משמעותי, ועד ללולאה הכי חשובה, שהעלתה בי התרשמות אמיתית מכל מה אשר הצלחתי להפיק בפרויקט זה.

אם היה לרשותי יותר זמן, הייתי מפתח פיצ'רים נוספים כפי שצינתי ב גרסאות המערכת, ועם סיום הפרויקט, אני שוקל להתחיל לפתח פרויקט חדש, או למקסם את הפרויקט המשחק הקיים.

אני מאמין שהפרויקט לימד אותי רבות על ניהול משאבים וזיכרון, ניהול קבצים, אלגוריתמיקה, גרפיקה, וכתיבת פעולות גמישות עבור משחקי מחשב בכללי, אך בנוסף לכך, גם עבור החשיבה בחיים בכלל, למדתי יותר על הדרך לפתרון בעיות. פיתוח ב Low Level דורש חשיבה "בלשית" ודיבוג של שעות ארוכות, מה שפיתח אצלי יכולת ניתוח מעמיקה, בנוסף להבנה לצורך לתכנן מקדים ודרך התכנון המקדים.

לסיום, אני שמח מאוד על ההזדמנות אשר קיבלתי, לעבוד על פרויקט זה ולהוציא אותו לפועל לבדיקה אמיתית. נהנתי מאוד בתהליך ולאחריו.

